

2025 年第十届“数维杯”大学生 数学建模挑战赛论文

题目 基于机器学习的“雨纷纷”动态预测与赏花路线优化模型

摘 要

在旅游业蓬勃发展的当下，清明节赏花活动已成为兼具文化传承与经济价值的重要载体。本文系统采集清明假期期间降雨量、日均温等多维气象数据，以及花卉所在地区的气象因素数据，经深度处理与分析，构建出综合性气象预测与旅游路线优化模型，旨在为游客出行提供精准决策支持。

本文基于题目给出网站进行数据收集工作，获取西安、吐鲁番、婺源、杭州、毕节、武汉、洛阳七个城市近 20 年清明假期期间降雨量，持续降雨时间等指标。并收集油菜花、杏花、杜鹃始期以及花期以及花卉所在的华东地区和西北地区的气象因素数据。基于收集的数据集进行预处理。

问题一中，为预测 2025、2026 年降雨量、降雨持续时间及判断“雨纷纷”状态，基于 7 地历史气象数据，用 **LSTM 神经网络**构建时序预测模型与**双任务学习框架**。结果显示 2026 年仅武汉、西安和婺源有“雨纷纷”现象。通过**卡尔曼滤波**结合实际数据修正模型，提升准确率与召回率分别为 84.9%和 85.7%，从而减小估计误差。

问题二中，为建立花卉预报模型，选取油菜花、杏花、杜鹃花，结合气象因子数据构建**有效积温模型**预测其始花期与花期时长，**经线性回归检验**后，对不满足的杏花、杜鹃花采用**随机森林**预测。结果显示，油菜花、杏花、杜鹃花在 2026 年的开花时间与花期分别为 2 月 24 日（194 天）、3 月 29 日（84 天）、3 月 23 日（22 天）。

问题三中，为科学规划清明赏花攻略。本文先依据问题一、二的数据划分华东地区和西北地区，规划婺源-杭州、西安 - 吐鲁番两条赏花路线；再构建四项指标，用 **AHP 法**确定权重，搭建多指标评价模型，以**高斯函数拟合**等方法提升准确度，最终华东路线评分 87.79 高于西北路线 37.56，实现 2026 年清明赏花攻略设计。

问题四中，针对城市赏花经济的经济效益，构建**多变量约束优化模型**，选取四个城市作为研究对象，协同优化门票价格、花期延长和政府投入。通过 **SLSQP 算法**求解，得出最优决策方案及效益提升成果，分析表明精准调控可增效，并提出推动赏花经济发展的相关建议。

关键词：LSTM 时间序列 卡尔曼滤波 有效积温模型 随机森林 优化模型 雨纷纷

目录

- 一、 问题重述 1
- 二、 问题分析 1
- 三、 模型假设 2
- 四、 符号说明 2
- 五、 数据预处理 3
 - 5.1 指标数据来源 3
 - 5.1.1 缺失值处理 4
 - 5.1.2 异常值处理 4
 - 5.1.3 数据标准化 5
- 六、 模型的建立与求解 5
 - 6.1 问题一模型的建立 5
 - 6.1.1 问题一模型的建立与求解 6
 - 6.1.2 LSTM 时间序列预测模型的结果 11
 - 6.1.3 LSTM 时间序列预测的模型修正 13
 - 6.2 问题二模型的建立 14
 - 6.2.1 积温基础模型的花卉预报模型构建 15
 - 6.2.2 随机森林的花卉预报模型构建 18
 - 6.2.3 预测结果的整合与分析 21
 - 6.3 问题三模型的建立与求解 22
 - 6.3.1 评价指标体系构建 23
 - 6.3.2 确定权重构建综合得分模型 24
 - 6.3.3 自由行攻略制定 26
 - 6.3.4 结论 27
 - 6.4 问题四模型的建立 28
 - 6.4.1 模型核心架构与目标函数 28
 - 6.4.2 决策变量与约束条件 28
 - 6.4.3 核心数学模型构建 29
 - 6.4.4 模型输出与策略分析 32
 - 6.4.5 赏花经济产业链报告 33
 - 6.5 模型的评价、改进与推广 34
 - 6.5.1 模型的优点 34
 - 6.5.2 模型的缺点 35
 - 6.5.3 模型的改进 36
 - 6.5.4 模型的推广 36
- 参考文献 37
- 附录.....33

一、 问题重述

问题一：“雨纷纷”天气的量化分析与预测：依据天气现象分类标准，明确“雨纷纷”对应的降雨量区间（如小雨、中雨量级）及持续时间范围，建立数学模型分析 2026 年清明假期西安、吐鲁番、婺源、杭州、毕节、武汉、洛阳七地的降雨概率。利用近 20 年历史天气数据验证模型合理性，并提出基于最新实况的动态修正方法。

问题二：春花花期预测模型构建：选取 2-3 种代表性花卉（如杏花、油菜花、牡丹），结合气象因子（气温、光照、降水等）与物候学原理，建立 2026 年开放时间 & 花期长度的预报模型，为赏花活动提供科学时间窗口。

问题三：清明踏青赏花自由行攻略制定：整合 2026 年清明假期天气预报与花期预测结果，综合考虑景点花期适宜度、天气舒适度、交通可达性等因素，设计兼具科学性与实用性的旅游路线，提升游客体验。

问题四：“赏花经济”产业链延长策略与效益评估：针对花期短暂的产业痛点，提出生物技术干预、空间布局优化、文化活动策划等具体措施，建立数学模型量化分析措施实施后对游客量、消费结构、经济收入的影响，为地方政府提供可持续发展建议。

二、 问题分析

针对问题一，基于 2005 年至 2024 年 7 个地区的历史气象数据，本文通过 LSTM 神经网络构建时序预测模型实现对“雨纷纷”天气现象的预测，搭建双任务学习框架，结合回归和分类任务，从而实现对降雨量和降雨持续时间的预测和对“雨纷纷”天气状态的条件判断。

针对问题二，基于 2015 年至 2024 年华东地区和西北地区的花期数据，本文通过有效积温模型、随机森林的方法来识别关键影响因子并构建开放时间 & 花期预报模型，最终实现对 2026 年目标花卉开放时间和花期的有效预报，为“科技赋能赏花”提供实践路径。

针对问题三，考虑赏花路线综合评价受多维度指标的非线性关联及权重配置问题影响，本文构建包含天气适宜度、距离便捷度、交通效率、花期重叠度的 4 项指标体

系，运用层次分析法确定权重并搭建多指标综合评价模型，实现对华东“婺源—杭州”路线和西北“西安—吐鲁番”路线的量化评分与路线优选，进而为 2026 年清明假期赏花自由行设计提供攻略。

针对问题四，考虑到城市赏花经济的精细化运营需求，本文基于上述内容的花期数据，构建门票价格、花期延长、政府投入多变量约束协同优化模型，实现四城市赏花经济总效益最大化，最终给地方政府提供一些具体措施，使“赏花经济”达到最佳效果。

三、 模型假设

1. 假设 7 个地区的地形、海拔、植被覆盖等地理环境因素对“雨纷纷”天气现象的影响可忽略不计，即模型仅通过历史气象数据便可捕捉天气规律。
2. 假设预测时段内的天气系统演变规律与历史数据中的时序模式一致，未出现突发的气候异常，如厄尔尼诺现象突变、极端寒潮等。
3. 假设 7 个地区的天气系统相互独立，不存在跨区域的气象要素传播，模型可对每个地区单独建模，无需考虑区域间的空间依赖关系。
4. 假设“雨纷纷”天气状态只影响游客出行意愿，即导致客流量减少，不影响其他变量。
5. 假设本文所选的代表花卉只受积温、日照时长、日均温、降雨量和土壤湿度影响。

四、 符号说明

符号定义	符号说明
σ	Sigmoid 函数
W	权重矩阵
i_t	输入门
f_t	遗忘门
$\tilde{C}_t$	候选细胞状态
o_t	输出门

GDD	积温 (°C · 天)
T _{mean}	日均温 (°C)
SUN	日照时长 (h)
P	降水量 (mm)
SWC	土壤湿度 (%)
FL	花期长度 (天)
SFS	始花期 (天)

五、数据预处理

5.1 指标数据来源

本文选取美国国家海洋、大气管理局和天气网发布的气象数据集，选取西安、吐鲁番、婺源等 7 个地区的气象站点，筛选出 2004 年至 2025 年的降雨量及降雨持续时间数据集。

由于气象网站问题，本文将无法直接搜索站点的城市中用其省内站点数据代替，如景德镇站点替代婺源地区。对于多个站点或站点有变动的地区，本文则按年份选取最优站点进行数据统计，如西安在 2011 年后使用泾河站点。详细见下表：

表 1 七个城市站点选取表

城市	编号	站点名称
西安	57036/57131	咸阳/泾河
吐鲁番	51573	吐鲁番
婺源	58527	景德镇
杭州	58457	萧山
毕节	57707	毕节
武汉	574940	天河
洛阳	57071	孟津

5.1.1 缺失值处理

由于个别年份的数据未公示导致指标中相关的数据存在缺失（如西安站点 23 年 4 月 5 日的降雨量），本文通过 Little's MCAR 检验发现，缺失数据符合完全随机缺失机制。并且通过正态 Q-Q 检验，发现数据在完整情况下近似服从正态分布，故可考虑采用贝叶斯多重插补法填补空缺值。

首先，将正态分布作为先验分布来描述缺失值的初始猜测。例如，根据对全国往年数据的初步分析，知道冷链覆盖率的均值为 μ_0 、标准差为 σ_0 ，那么可以设定先验分布为 $N(\mu_0, \sigma_0)$ 。

其次，利用观测数据 y_{obs} 和缺失值 y_{mis} 组成的完整数据集，根据多元正态分布的性质， $f(y_{obs}, y_{mis})$ 的联合概率密度函数为

$$f(y_{obs}, y_{mis}) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (y - \mu)^T \Sigma^{-1} (y - \mu) \right] \quad (1)$$

再对其进行积分，得到观测数据的边缘概率密度函数，即似然函数

$$L(\mu, \Sigma | y_{obs}) = \int f(y_{obs}, y_{mis}) dy_{mis} \quad (2)$$

最后，通过贝叶斯公式计算后验分布中随机抽取样本作为插补值。

5.1.2 异常值处理

由于每年数据的日期存在不同格式与前导零问题，本文对无数据缺失的指标进行正则性表达式预处理，从而统一格式（如 4 月 5 日）使得模型可以正常识别日期，进而防止模型学习错误时间规律。以下为异常值过滤模块执行效果：

表 2 异常值过滤模块执行表

问题类型	示例数据	处理方式
格式错误	"4 月 5 日" → 无年份	结合年份列补全
数值超限	2023-02-30（不存在）	标记为 NaT 后删除
前导零问题	"04-05" → 4-5	lstrip('0')处理

5.1.3 数据标准化

由于 LSTM 神经网络中，权重影响模型自动学习的参数矩阵，而为了消除降雨量和降雨持续时间对权重的量纲影响，因此本文采用 Z - Score 标准化，对特定特征的数据集，计算该特征的均值 μ 和标准差 σ ，再将 4 个环节数据集的每个数据 z_i 通过 Z - Score 标准化公式进行转换：

$$Z_i = \frac{X_i - \mu}{\sigma} \quad (3)$$

以此消除量纲影响，消除数值大小对权重的影响。

六、模型的建立与求解

6.1 问题一模型的建立

由于各地天气系统具有明显的时序相关性、变量间的非线性关系和非平稳性，而 LSTM 时间序列预测模型可通过门控机制（输入门、遗忘门、输出门）能够有效捕捉长序列中的依赖关系，进而实现动态学习和预测。

基于 7 个地区的历史气象数据，本文通过 LSTM 神经网络构建时序预测模型实现对“雨纷纷”天气现象的预测，搭建双任务学习框架，结合回归和分类任务实现对降雨量和降雨持续时间的预测，同时结合 MSE 损失和 BCE 损失平衡数值预测和分类精度，最后进行“雨纷纷”状态的条件判断。

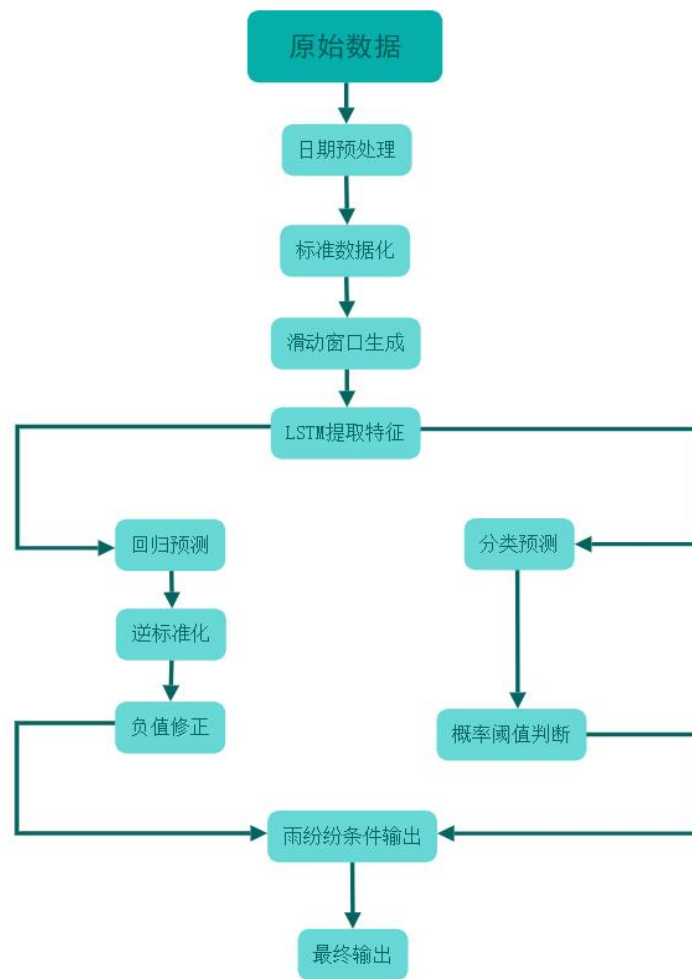


图 1 LSTM 时间序列预测模型思路

6.1.1 问题一模型的建立与求解

根据中国气象局降水等级标准, 24 小时降水量 0.1-9.9mm 为小雨, 10-24.9 mm 为中雨, 同时 3 小时的持续时间既能确保降水过程的连贯性, 也符合层状云降水的典型持续时长。因此, 本文划定“雨纷纷”的降水量为 0.1-10mm, 同时满足当天降雨持续 3 小时以上, 从而覆盖了小雨的全部范围及中雨的下限, 与稳定性降水的弱强度特征高度吻合, 也符合稳定性降水的时间特征。

$$\text{RainFlag}^{(m)} = \begin{cases} 1, & \text{if } 0.1 \leq \hat{y}_{\text{precip}}^{(m)} \leq 10.0 \wedge \hat{y}_{\text{dur}}^{(m)} \geq 3 \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

(1) 数据输入

设测试地区的气象时序观测数据为 D ， $d=2$ 为特征维度，即分支个数：

$$D = \{\mathbf{X}_t\}_{t=1}^T \subset \mathbb{R}^{T \times d} \quad (5)$$

模型建立从历史序列到未来状态的映射关系为：

$$D = f_{\theta}: \{\mathbf{X}_{t-k}\}_{k=0}^{n-1} \mapsto \{\mathbf{Y}_{t+m}\}_{m=1}^M \quad (n=3, M=6) \quad (6)$$

- $\mathbf{X}_{t-k}$ 为历史窗口
- $\mathbf{Y}_{t+m}$ 为预测窗口

以此使用微分方程描述记忆单元演化过程。

(2) 动力学方程

再次，采用 LSTM 的动力学方程描述其核心门控机制（输入门、遗忘门、输出门）对细胞状态和隐藏状态的更新过程，进而实现长期依赖记忆的预测。本文采用了门控循环单元（GRU）模型，以捕捉气象数据中时序依赖特征。GRU 模型数学表达如下：

$$\begin{cases} f_t = \sigma(W_f[h_{t-1}; x_t] + b_f) & (\text{遗忘门控制长期记忆衰减}) \\ i_t = \sigma(W_i[h_{t-1}; x_t] + b_i) & (\text{输入门调节新信息注入}) \\ \tilde{C}_t = \tanh(W_c[h_{t-1}; x_t] + b_c) & (\text{候选状态生成}) \\ C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t & (\text{细胞状态更新}) \\ h_t = o_t \odot \tanh(C_t) & (\text{隐状态输出}) \end{cases} \quad (7)$$

其中， σ 为 Sigmoid 函数， W 为权重矩阵， b 为偏置， i_t 为输入门， f_t 为遗忘门， $\tilde{C}_t$ 为候选细胞状态， o_t 为输出门。

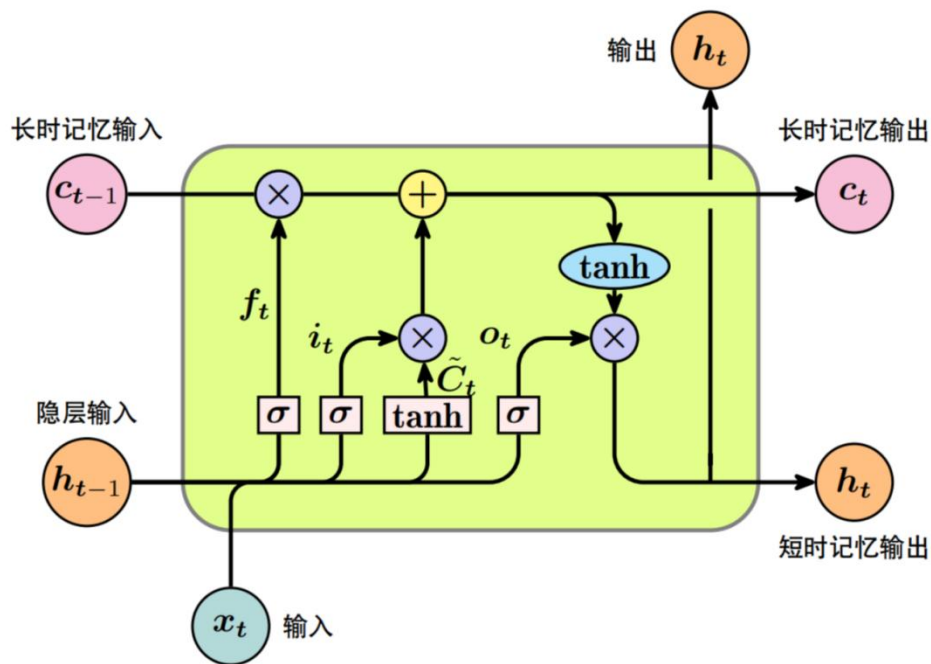


图 2 LSTM 门控机制图

(3) 滑动窗口

滑动窗口是一种数据预处理技术，将长序列分割为固定长度的重叠或非重叠子序列，用于处理序列数据的局部上下文关系。结合 LSTM 时，窗口长度通常对应时间步数，窗口滑动步长决定输入序列的重叠程度。设原始序列为 $X = \{x_1, x_2, \dots, x_N\}$ ，窗口大小为 K ，步长为 S ，则第 t 个窗口的输入为：

$$X_t = \{x_t, x_{t+1}, \dots, x_{t+K-1}\} \quad (8)$$

• 其中 $t \in [1, N - K + 1]$

若用于预测下一时刻值 x_{t+K} ，则输入 LSTM 的样本为 (X_t, x_{t+K}) ，通过 LSTM 层计算隐藏状态与细胞状态，进而将长序列转换为固定长度的子序列，适配 LSTM 的输入维度，实现降维与局部特征提取；同时使得每个窗口包含前 $K-1$ 个时刻的信息，用于预测当前时刻。



图 3 LSTM 移动窗口机理图

本文以 2005 年至 2024 年清明假期 3 天的数据作为输入，设时间步长为 $T=1$ 循环然后通过循环变量 i 依次递增 1，使得窗口每次向前滑动 1 天，捕捉天气系统的短期变化规律，而输出窗口为 2025 年和 2026 年清明假期各 3 天的预测数据。

(4) 双分支解码器

此外，由于模型需对天气进行回归和分类，本文构建多任务学习框架，首先，采用回归分支全连接网络，输出降水量和持续时间的连续值；其次，采用分类分支 Sigmoid 激活输出每日“雨纷纷”概率，进行“雨纷纷”状态的条件判断。

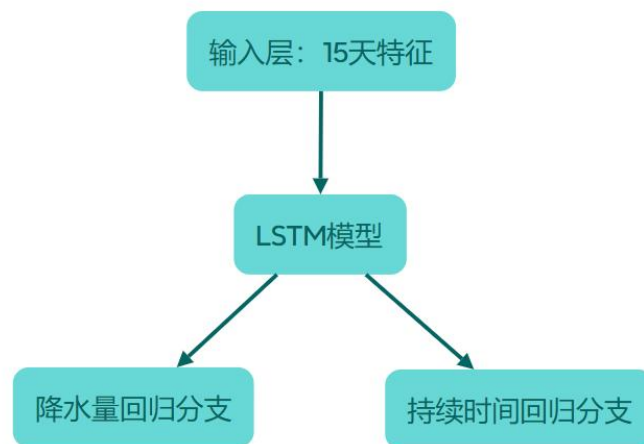


图 4 LSTM 移动窗口机理图

双分支解码器是一种模型架构设计，通常在序列生成任务中使用，其核心是通过两个并行的分支路径处理不同类型的信息或特征，最终融合输出解码结果。结合 LSTM 时，两个分支可能分别对应 LSTM 的隐藏状态 (Hidden State) 和细胞状态 (Cell State)，或处理不同层级的输入数据。

针对两个任务，本文引入可同时服务于两个任务的 LSTM 层时空特征，共享同一组 LSTM 的最终隐状态，即双分支解码器。其中，回归分支：全连接网络将 LSTM 提取的时序特征映射为 2025 年和 2026 年各 3 天的降水量和持续时间的连续值；基于相同时序特征，独立计算每日“雨纷纷”状态的二元概率估计，实现“雨纷纷”判断。

第一，通过两层非线性变换，将 64 维隐状态映射到气象要素空间。

$$z_{\text{reg}} = W_{\text{reg}2} \cdot \text{ReLU}(W_{\text{reg}1} \cdot H_n) \in \mathbb{R}^{M \times 2} \quad (9)$$

- $W_{\text{reg}1} \in \mathbb{R}^{32 \times h}, W_{\text{reg}2} \in \mathbb{R}^{2M \times 32}$ 为回归分支参数

第二，设定阈值 0.5，当 z_{cls} 大于 0.5 判断为第 m 天为"雨纷纷"。

$$z_{\text{cls}} = \sigma(W_{\text{cls}2} \cdot \text{ReLU}(W_{\text{cls}1} \cdot H_n)) \in [0,1]^M \quad (10)$$

- $W_{\text{cls}1} \in \mathbb{R}^{32 \times h}, W_{\text{cls}2} \in \mathbb{R}^{M \times 32}$ 为分类分支参数

由于"雨纷纷"的判定不仅依赖降水量与持续时间的精确数值预测，还需结合气象学规则进行状态分类，因此本文构建融合回归与分类误差的混合损失函数，计算回归分支和分类分支的损失，进而确保降雨参数的物理真实性和强化气象标准与预测结果的逻辑一致性。以下为联合损失 $\mathcal{L}$ 计算公式：

$$\mathcal{L} = \mathbb{E}_{(x,y)} [\|y_{\text{reg}} - \hat{y}_{\text{reg}}\|^2 + \lambda \mathcal{H}(y_{\text{cls}}, \sigma(\hat{y}_{\text{cls}}))] \quad (11)$$

- $\mathbb{E}$ 为期望计算，实际实现为批量平均
- σ 为 Sigmoid 函数（隐含于分类输出）
- λ 为模型参数，测定为 1 时优化效果最佳

(5) 数据逆标准化与约束嵌入

由于需直观对比 2025 年降水量与持续时间的预测值与实际值，本文对 LSTM 时间序列预测模型的数据集进行反标准化运算，恢复其原始量纲，进而便于模型后续模型的检测。以下为具体计算公式：

$$\hat{Y}_{\text{raw}} = \hat{z}_{\text{reg}} \odot \sigma + \mu \quad (12)$$

最后，由于 LSTM 模型特征外推导致的预测数据的“越界”，即测试数据的特征值超出训练数据范围时，模型可能在未知区域外推导致预测值为负，本文通过物理约

束嵌入，用 0 值修正降水量与持续时间的异常预测值。

6.1.2 LSTM 时间序列预测模型的结果

通过 LSTM 时间序列预测模型的回归分支计算得到 7 个地区的 2025 年和 2026 年降雨量和降雨持续时间，再结合分类分支判断得到“雨纷纷”天气情况，以毕节地区为例，其余详见附件：

表 3 毕节地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	0.00	3.08	否
2025 年 4 月 5 日	0.11	0.18	否
2025 年 4 月 6 日	0.32	2.92	否
2026 年 4 月 4 日	0.13	0.5	否
2026 年 4 月 5 日	0.05	1.48	否
2026 年 4 月 6 日	0.00	1.72	否

此外，本文定义 2026 年清明假期会出现“雨纷纷”为 3 天中若预测出某天出现降水量为 0.1-10mm，并满足当天降雨持续 3 小时以上的情况，即判断为“雨纷纷”状态。

表 4 2026 年各地区“雨纷纷”状态

地区	“雨纷纷”
毕节	否
杭州	否
洛阳	否
吐鲁番	否
武汉	是
婺源	是
西安	是

分析上表可知，2026 年清明假期只有武汉和西安地区会出行“雨纷纷”现象，其余 5 个地区均不会出现“雨纷纷”。

为检验模型合理性，本文引入召回率、准确率、平均绝对误差和 R 方对模型进行检测。如下表：

表 5 2025 年降雨量检测表

平均绝对误差	R 方	准确率	召回率
0.066	81.2%	61.9%	85.7%

召回率（Recall）是指在所有实际为正例的样本中，被模型正确预测为正例的比例。本文设置 TP 表示真正例，即预测为“雨纷纷=True”且实际为“雨纷纷 = True”的样本数；FN 表示假负例，即预测为“雨纷纷 = False”但实际为“雨纷纷 = True”的样本数。

召回率主要衡量的是模型捕捉正例的能力，值越高说明模型对正例的识别能力越强。在气象预测“雨纷纷”这一情境中，其计算公式为：

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (13)$$

准确率（Accuracy）是指模型正确预测的样本数占总样本数的比例。本文设置 TP 为真正例；TN 为真负例，即预测为“雨纷纷 = False”且实际为“雨纷纷 = False”的样本数；FP 为假正例，即预测为“雨纷纷 = True”但实际为“雨纷纷 = False”的样本数；FN 为假负例。

准确率反映了模型对所有样本的整体预测正确程度，取值范围在 0 到 1 之间，值越高表示模型的预测性能越好。在气象预测的例子中，其计算公式为：

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (14)$$

此外，当实际没有正例时，即清明假期 3 天均无“雨纷纷”现象，召回率的严格

数学计算会遇到分母为 0 的问题，此时通常约定召回率为 1（而非 0），因为模型没有漏判任何需要召回的正例。

通过分别计算 2025 年 7 个地区召回率和准确率，结果如下表：

表 6 各地区准确率及召回率

地区	召回率	准确率
毕节	100%	100%
杭州	100%	0%
洛阳	100%	100%
吐鲁番	100%	100%
武汉	100%	66.7%
婺源	0%	0%
西安	100%	66.7%

为验证 LSTM 模型的合理性，本文分别计算 2025 年 7 个地区的召回率和准确率均值，结果如下表：

表 7 LSTM 模型检测表

准确率	召回率
61.9%	85.7%

6.1.3 LSTM 时间序列预测的模型修正

通过 LSTM 模型得到 2025 年的预测值序列，同时通过模型接口读取存在 2025 年的实际观测数据，并使用卡尔曼滤波修正后续预测。本文通过“预测—用实测修正—更新不确定性”的循环，使 2025 年预测值在噪声中不断逼近真实值，利用卡尔曼增益动态分配预测与实测的信任度，最小化估计误差。

首先，根据 2025 年实际值的最优估计，预测当前数值状态，使得设预测值与实际估计值趋势一致；其次，采用实测数据 z_t 修正预测值，假设修正的状态变量是实际值 x_t ，模型预测值为 $\hat{x}_{t|t-1}$ ，实际观测值为 z_t 。

$$K_t = \frac{P_{t|t-1} \cdot H^T}{H \cdot P_{t|t-1} \cdot H^T + R} \quad (15)$$

其次，计算卡尔曼增益 K_t ，测算决定实测值对修正的贡献度

$$z_t = H \cdot x_t + v_t \quad (16)$$

- $P_{t|t-1}$ 为预测阶段的误差协方差
- R 为观测噪声协方差

然后用卡尔曼增益融合预测值 $\hat{x}_{t|t-1}$ 和实测值 z_t ，得到修正后的状态估计 $\hat{x}_{t|t}$ ：

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t \cdot (z_t - H \cdot \hat{x}_{t|t-1}) \quad (17)$$

- $\hat{x}_{t|t-1}$ 为阶段 1 的预测值
- $-H \cdot \hat{x}_{t|t-1}$ 为观测残差，即实测值与预测值的差异

为验证卡尔曼滤波修正效果，本文分别再计算 2025 年 7 个地区的召回率和准确率均值，结果如下表：

表 8 LSTM 模型检测表

准确率	召回率
84.9%	85.7%

分析上表可知，模型召回率为 85.7%，LSTM 模型对“雨纷纷”天气辨认敏感性高，即模型能正确识别出 85.7% 的情况，漏报率极低而体现其高实用性；而准确率为 84.9% 说明 LSTM 模型对“雨纷纷”天气预测整体区分能力较强，但可能天气数据难免受季节、地域影响，导致 LSTM 时间序列对部分地区预测存在偏离。

6.2 问题二模型的建立

在问题二中，本文选取了油菜花、杏花和杜鹃花作为代表性花卉进行分析，结合

收集到的气象因子数据，首先通过物候学的有效积温模型对花卉的始花期和花期时长进行预测分析，其次对其进行线性回归检验，然后对不满足条件的杏花和杜鹃花进行随机森林预测分析，最后计算各花卉的开花时间，进而帮助游客科学规划赏花行程。

6.2.1 积温基础模型的花卉预报模型构建

积温是指某一时段内逐日平均气温累加之和。有效积温模型是物候学的相关理论，有效积温是生物能较好地表征作物发育进程，本问以有效积温的时间用以计算花卉的始花期，再通过模型公式计算花卉的花期长度。

模型的构建（以油菜花为例）：

基于物候学理论，用积温计算始花期，计算公式如下：

$$\text{始花期} = a + b \cdot \text{积温} \quad (18)$$

通过历史油菜花的始花期数据和环境积温，利用线性回归（最小二乘法）确定始花期，计算出 a 和 b 的最优值，使得预测值和实际值的均方误差 MSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - (a + b \cdot x_i))^2 \quad (19)$$

- X_i 为第 i 年的积温；
- Y_i 为第 i 年的始花期，以 1 月 1 日为第 1 天

$$b = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad a = \bar{y} - b \cdot \bar{x} \quad (20)$$

- $\bar{x}$ 为积温
- $\bar{y}$ 为始期的平均值

根据以上算法，计算积温与始花期的线性回归公式：

$$\text{SFS} = -0.143 \times \text{GDD} + 119.607 \quad (21)$$

花期长度与日均温有关，结合相关资料得到花期长度与日均温的公式如下：

$$FL = c + d \times T_{\text{mean}} \quad (22)$$

计算截距和斜率的方法同上(最小二乘法)，得到对于日均温（℃）与花期（天）的线性回归公式：

$$FL = 10.5139 \times T_{\text{mean}} + 52.917 \quad (23)$$

模型分析表如下：

表 9 油菜花始花期与积温线性回归模型结果分析表

	B	标准误	t	p	R ²	F
常温	119.607	9.991	11.972	0.000***	0.808	F=33.565
积温	-0.144	0.025	-5.794	0.000***		P=0.000***

注：***、**、*分别代表 1%、5%、10%的显著性水平

该线性回归模型基于 10 个样本数据，以积温为自变量、始花期为因变量进行建模。结果显示模型整体显著（F=33.565，p<0.001），解释力较强（R²=0.808），模型结果符合要求，

花期长度与日均温线性回归模型结果分析表如下：

表 10 油菜花期长度与日均温线性回归模型结果分析表

	B	标准误	t	p	R ²	F
常温	52.918	14.975	3.534	0.008***	0.905	F=76.333
日均温	10.514	1.203	8.737	0.000***		P=0.000*

注：***、**、*分别代表 1%、5%、10%的显著性水平

可见，以上两个线性回归模型均通过检验，模型构建良好。

同理，将杏花与杜鹃花的始花期和花期长度以及其相关的气象因子的历史数据，利用最小二乘法，得到对应的始花期公式和花期长度公式，公式如下：

杏花的积温与始花期的线性回归公式：

$$SFS = 0.0686 \times GDD + 63.767 \tag{24}$$

杏花日均温与花期长度的线性回归公式：

$$FL = 4.3303 \times T_{mean} + 34.146 \tag{25}$$

杜鹃花的积温与始花期的线性回归公式：

$$SFS = 0.0917 \times GDD + 54.628 \tag{26}$$

杜鹃花日均温与花期长度的线性回归公式：

$$FL = -2.3525 \times T_{mean} + 89.6039 \tag{27}$$

杏花和杜鹃花线性回归模型结果分析如下：

表 11 杏花和杜鹃花线性回归模型结果分析

花卉种类	R_1^2	F_1	R_2^2	F_2
杏花	0.119	$F=1.219$ $P=0.298$	0.176	$F=1.918$ $P=0.199$
杜鹃花	0.077	$F=0.664$ $P=0.439$	0.975	$F=317.853$ $P=0.000^{***}$

注：***、**、*分别代表 1%、5%、10% 的显著性水平，类别 1 为始花期与积温线性回归模型，类别 2 为花期长度与日均温线性回归模型

由上表可见，在始花期与花期长度的线性回归分析中，杏花的始花期和花期长度均未通过显著性检验，其中杜鹃花的始花期模型同样不满足线性假设。模型中使用的变量存在非线性关系，不适用于线性模型，下文将使用随机森林机器学习的方法预测非线性类数据的结果。

本文利用“历史趋势外推法”的方法，通过 python 工具结合历史数据统计规律生成合理的假设输入，预测得到了 2026 年的 5 个气象因子的预测数据，结果如下表：

表 12 2026 年气象因子数据

气象因子	积温 GDD	日均温	日照时长	降水量	土壤湿度
油菜花	447.75	13.475	250.5	77.75	74.25
杏花	434.2	12.14	209	46.2	68.2
杜鹃花	334	28.85	198.5	59	76

将预测 2026 年与花卉的始花期和花期长度的气象因子结合，利用积温模型可得到油菜花的 2026 年的始花期和花期长度，结果如下表：

表 13 积温模型结果表

花卉种类	始花期	花期长度
油菜花	55	194

6.2.2 随机森林的花卉预报模型构建

基于上文收集到的数据，本文将利用与花卉花期有关的 5 个气象因子，构建随机森林回归模型预测花卉的始花期和花期长度。最终通过结果分析揭示不同环境因子在特定条件下对花期的非线性影响机制，为花卉物候预测提供理论依据与技术支持。

随机森林回归是在生成众多决策树的过程中，是通过对建模数据集的样本观测和特征变量分别进行随机抽样，每次抽样结果均为一棵树，且每棵树都会生成符合自身属性的规则和判断值，而森林最终集成所有决策树的规则和判断值，实现随机森林算法的回归。

本文构建了两个随机森林模型，分别用预测始花期时间和预测花期长度时间，分别用于预测杏花和杜鹃花的始花期和花期长度。

模型的构建过程如下：

(1) 数据准备

首先需要准备用于训练和测试模型的数据集。数据集应包含特征和对应的目标变量。本文特征变量选用 5 个气象因子分别为积温 GDD、日均温、日照时长、降水量、土壤湿度，目标变量为花卉的始花期和花期长度。将数据集按比例划分为训练集和测试集，其中训练集用于训练模型，测试集用于评估模型的性能。

(2) 构建随机森林

本文构建随机森林回归模型，通过可设置参数来控制随机森林的行为，例如决策树的数量、特征选择的方式、决策树的生长方式等。可以根据实际问题 and 需求进行参数的调整，详见附录。

(3) 训练模型

使用训练集对随机森林回归模型进行训练。模型将根据训练集中的样本和目标变量的值来构建多棵决策树，并在每棵树上进行特征选择和划分。

(4) 预测结果

使用训练好的随机森林回归模型对测试集中的样本进行预测。模型将对每棵决策树的预测结果进行平均或加权平均，从而得到最终的回归预测结果。

(5) 模型评估

通过与真实目标变量的比较，评估模型的性能。可以使用各种回归性能指标，例如均方误差、平均绝对误差、决定系数和交叉验证 MAE 来评估模型的准确性和泛化能力。

(6) 模型调优

构建模型的评估机构，可以尝试调整随机森林的参数，提高模型的性能。

(7) 模型应用

在模型评估和调优后，可以使用训练好的随机森林回归模型进行实际的预测。可以预测的特征向量数据，即表 2026 年气象因子数据输入到模型中，从而计算出回归预测结果，得到需要预测的花卉始花期和花期长度。

本文检测有效积温模型预测数据，计算得到检验结果如下表：

表 14 始花期模型检验表

	MAE (天)	RMSE	R^2	交叉验证 MAE
杏花	1.89	2.05	0.94	2.34
杜鹃花	1.20	1.34	0.98	1.67

表 15 花期长度模型检验表

	MAE	RMSE	R^2	交叉验证 MAE
杏花	1.40	1.67	0.99	2.03
杜鹃花	0.80	0.98	0.99	1.21

可见所有模型的 R^2 值均接近 1，表明模型拟合效果良好。交叉验证 MAE 值较小，说明模型具有较好的泛化能力杜鹃花花期预测模型表现最佳 ($R^2=0.99$, MAE=0.80 天)。整体而言，杏花和杜鹃花的始花期预测模型和花期长度预测都具有可信度，可用于预测始花期花期长度

将预测的气象因子数据放入随机森林回归模型中，计算得到花卉的始花期和花期长度，模型运行结果如下：

表 16 随机森林预测结果表

花卉种类	始花期	花期长度
------	-----	------

杏花	88	84
杜鹃花	82	22

可见，以上三朵花卉的花期结束时间均在清明节假期之后，而杜鹃花的花期较短，花期结束时间大约在 4 月 15 日左右，油菜花和杏花的花期较长，花期结束时间在下半年。

6.2.3 预测结果的整合与分析

有效积温模型能从物候学方面科学的解释作花卉长与热量积累的关系，通过线性模型解释花卉的始花期和花期长度与气象因子之间的线性关系，为花卉的生长提供较为可靠的基础预测。随机森林模型通过从海量数据中提取特征，能够有效捕捉复杂的非线性关系，并对数据中的噪声和异常值展现出较强的鲁棒性，适合用于分析气象因子对花卉花期的非线性影响。

结合有效积温模型和随机森林回归模型，预测得到了三种花卉的始花期和花期长度：

表 17 花卉花期预测结果表

花卉种类	始花期	花期长度	开花日期	末花期
油菜花	55	194	2 月 24 日	9 月 21 日
杏花	88	84	3 月 29 日	6 月 21 日
杜鹃花	82	22	3 月 23 日	4 月 15 日

本文通过融合有效积温模型与随机森林回归算法，构建了花卉花期预测系统。结果表明，油菜花在累积 55 日有效积温后启动花芽分化，于 2 月 24 日进入始花期，经 194 天营养生长完成生殖周期，至 9 月 21 日完成终花过程，其花期持续时长显著高于杏花和杜鹃花。杏树开花为 3 月 29 日，经历 84 天生殖发育后于 6 月 2 日完成落花过程；而杜鹃花在 3 月 23 日开花，仅维持 22 天短期花期即于 4 月 15 日结束。

有效积温模型基于温度累积效应量化花卉生长发育进程，随机森林回归模型则通过非线性拟合解析复杂气象因子的耦合影响，二者成功实现了对油菜花、杏花及杜鹃花花期的精确预测，为游客的赏花之旅提供了坚实的科学依据。

6.3 问题三模型的建立与求解

基于问题一和问题二分析，本文发现问题二所选取的代表花卉主要分布于华东和西地区，即涵盖西安、吐鲁番、婺源和杭州四个城市，因此问题三将围绕此四个城市进行分析。

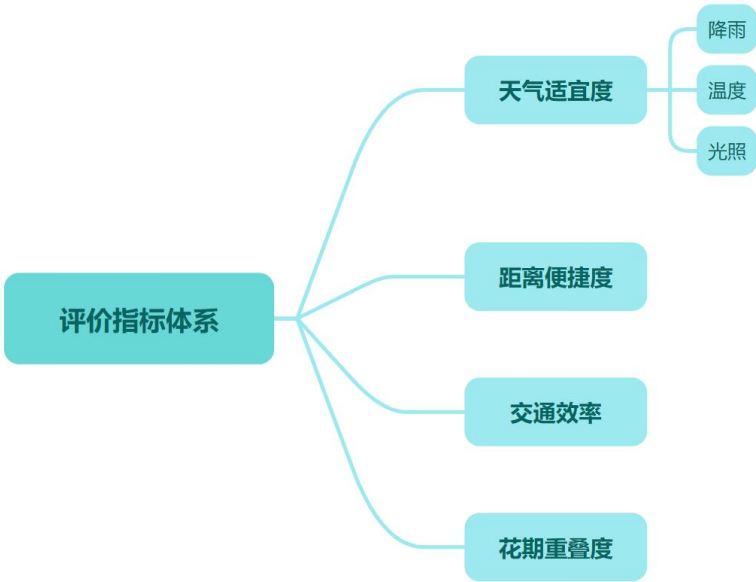


图 5 赏花路线综合评价模型

由于赏花路线的综合评价涉及多维度指标的非线性关联与权重配置，本文通过构建天气适宜度、距离便捷度、交通效率、花期重叠度共 4 项的指标体系，采用层次分析法 AHP 确定权重，搭建多指标综合评价模型，实现对华东路线：婺源—杭州，其代表花卉为杏花、杜鹃花和西北路线：西安—吐鲁番，其代表为花卉：油菜花，量化评分与路线优选，进而对 2026 年清明假期赏花自由行设计攻略。



图 6 赏花路线示意图

6.3.1 评价指标体系构建

(1) 对于天气适宜度评分 W ，其由综合降雨、温度、光照三要素构成，采用加权线性组合模型：

$$W = 0.4W_P + 0.3W_T + 0.3W_L \quad (28)$$

首先，参考气象学降雨等级标准^[1]对降雨量分段赋值：无雨或小雨 100 分；中雨 60 分；大雨 30 分，计算降雨评分 W_P 。

然后，基于高斯函数拟合人体舒适温度分布^[2]计算温度评分 W_T ：

$$W_T = 100 \cdot e^{-\left(\frac{T_{\text{avg}} - 20}{5}\right)^2} \quad (29)$$

其中 20°C 为最佳观赏温度，标准差 5°C 反映舒适区间的对称性。

最后，通过线性转换 4-12 小时有效光照时长^[3]测算光照评分 W_L ：

$$W_L = \begin{cases} 0 & L < 4 \\ \frac{L-4}{12-4} \times 100 & 4 \leq L \leq 12 \\ 100 & L > 12 \end{cases} \quad (30)$$

(2) 对于距离便捷度评分 D ，本文基于哈弗辛公式计算球面距离后，采用线性归一化：

$$D = 100 - \frac{D_{\text{km}} - 100}{1000 - 100} \times 60 \quad (31)$$

其中 100km 为短途阈值，1000km 为长途阈值，距离越短得分越高。

(3) 对于交通效率评分 T ，本文以高铁时速 250km/h 计算交通时间，线性归一化时间成本：

$$T = 100 - \frac{t_{\text{小时}} - 1}{6 - 1} \times 80 \quad (32)$$

其中，1 小时为短途时间基准，6 小时为长途时间上限。

(4) 对于花期重叠度评分 F ，本文根据清明假期每年的 4.4-4.6 日，即第 94-96 天，与花卉盛花期的重合天数分段赋值：

$$F = \begin{cases} 0 & \text{重合 0 天} \\ 40 & \text{重合 1 天} \\ 70 & \text{重合 2 天} \\ 100 & \text{重合 3 天} \end{cases} \quad (33)$$

6.3.2 确定权重构建综合得分模型

(1) 层次分析法（AHP）

本文通过专家经验法构建指标重要性两两比较矩阵 A ，反映决策者对各指标的偏好：

$$A = \begin{pmatrix} 1 & 2 & 3 & 1 \\ 1/2 & 1 & 2 & 1/2 \\ 1/3 & 1/2 & 1 & 1/3 \\ 1 & 2 & 3 & 1 \end{pmatrix} \quad (34)$$

其中，将天气评分（行 1）比距离评分（列 2）“稍微重要”（赋值为 2），比交通时间（列 3）“明显重要”（赋值为 3），与花期重叠度（列 4）“同等重要”（赋值为 1）。

(2) 权重计算与一致性检验

利用特征值法求解权重向量 w ，最大特征值 $\lambda_{\max} = 4.12$ ，对应权重：

$$w = [0.3832, 0.1915, 0.1051, 0.3202] \quad (35)$$

计算一致性比率得到 $CI = 0.04$ ， $RI = 0.90$ ，通过一致性检验。

(3) 综合得分模型

通过加权汇总各指标得分，计算得到， $w_W = 0.3832$ 、 $w_D = 0.1915$ 、 $w_T = 0.1051$ 、 $w_F = 0.3202$ ，则 AHP 确定的权重公式为：

$$S = w_W \cdot W + w_D \cdot D + w_T \cdot T + w_F \cdot F \quad (36)$$

(4) 模型实现与数据验证

本文输入气象数据和地理数据，通过综合评价模型计算出两条赏花线路，具体如下表：

表 18 毕节地区天气预测表

指标	华东路线	西北路线	权重
----	------	------	----

天气评分	71.707	17.464	0.3832
距离评分	88.658	-24.304	0.1915
交通时间评分	98.712	-9.732	0.1051
花期重叠度	100	100	0.3202
综合得分	87.79	35.56	—

由综合得分结果可知，华东路线因天气适宜、交通便捷，综合得分显著优于西北路线。

以下是评分可视化：

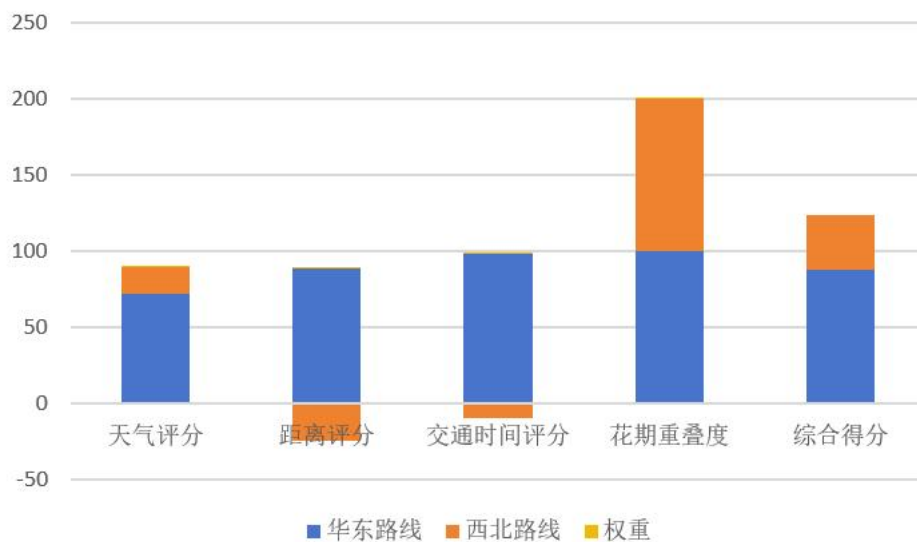


图 7 评分示意图

6.3.3 自由行攻略制定

(1) 华东线：婺源（油菜花）→ 杭州（杜鹃花）

第一天上午：江岭梯田

- 10 万亩梯田油菜花（始花期 55 天）
- 早上 6:30-8:00：无人机航拍光照评分 75.8 分的“梯田花海+徽派村落”全景

第一天下午：篁岭古村

- 古村晒台与油菜花田同框，可拍摄人物美景，日均温 12.14℃，舒适徒步

第二天上午：杨公堤

- 5 公里杜鹃景观带（始花期 82 天，清明进入盛花期）
- 摄影捕捉杜鹃花蕊细节，搭配西湖水面倒影，小雨天气可以增强取景质量。

第二天下午：太子湾公园

- 杜鹃与郁金香混植花境，利用清明 7.46 小时日照（无强光）拍摄写真。

(2) 西北线：西安（杏花）→ 吐鲁番（杏花）

第一天上午：蓝田华胥镇（西安）

- 万亩野生杏林（始花期 88 天，清明为盛花初期）
- 记录杏花与黄土高原沟壑的对比，无人机航拍 “杏林 + 梯田”

第一天下午：大明宫遗址（西安）

- 唐代宫殿遗址杏花坡，拍摄“丹凤门遗址和粉白杏花”古今同框

（日均温 13.47℃，适宜户外）

第二天全天：托克逊南湖村（吐鲁番）

- 早上 7:00 发车西安→吐鲁番高铁，建议携带颈枕、眼罩应对 8 小时车程
- 戈壁野杏林（观赏西北杏花，花期 84 天）
- 多风环境拍摄杏花与灰褐色戈壁，营造动感民族氛围

6.3.4 结论

根据评分模型可知，华东路线总得分 87.79 远高于西北路线 35.56，其差异源于华东线为短途交通和湿润气候，对比西北线为长途交通和干燥低温。综上所述，华东线凭借气象条件和交通效率成为最优解，而西北线更适合偏好“单一花卉+西北地貌”的深度游旅客。

6.4 问题四模型的建立

针对城市赏花经济的精细化运营需求，本文构建多变量约束优化模型，通过门票价格、花期延长、政府投入的协同优化，实现四城市（西安、吐鲁番、婺源、杭州）赏花经济总效益最大化。模型融合价格弹性理论、花期调控技术限制与财政预算约束，为区域旅游经济政策制定提供量化决策工具。

6.4.1 模型核心架构与目标函数

(1) 目标函数

定义总经济效益为四城市赏花经济收益减去政府基础设施投入，以亿元为单位：

$$\max \sum_{c=1}^4 (R_c + S_c - I_c) \quad (37)$$

- c 为城市索引（西安、吐鲁番、婺源、杭州）
- R_c 为城市 c 门票收入
- S_c 为城市 c 衍生收入
- I_c 为城市 c 政府基础设施投入

(2) 经济效益构成

- **门票收入：**受价格调整与游客量变化影响，与价格弹性直接相关。
- **衍生收入：**依赖游客停留时间，通过花期延长提升吸引力。
- **政府投入：**受财政预算约束，需在可行区间内优化配置。

6.4.2 决策变量与约束条件

(1) 决策变量（四城市 ×3 变量，共 12 个变量）

门票价格调整率(α_c):

$$\alpha_c \in [-0.3, 0.5] \quad (38)$$

即价格下调 30%至上调 50%。

花期延长天数(ΔD_c):

$$\Delta D_c \leq \Delta D_c^{\max} = \begin{cases} 15 & \text{西安、吐鲁番(杏花)} \\ 20 & \text{婺源(油菜花)} \\ 18 & \text{杭州(杜鹃花)} \end{cases} \quad (39)$$

政府投入(I_c , 亿元):

$$I_c \in [I_c^{\min}, I_c^{\max}] = \begin{cases} [12, 25] \\ [6, 15] \\ [15, 30] \\ [25, 40] \end{cases} \quad (40)$$

(2) 政府投入约束

$$I_c \in [I_c^{\min}, I_c^{\max}] \quad (41)$$

其中，投入下限为当前投入水平，上限为当前投入的 3 倍或总预算的 50%（取较小值）。

6.4.3 核心数学模型构建

(1) 游客数量模型基于价格弹性与投入吸引力的游客量动态调整公式:

$$N_c = N_c^{(0)} \left(1 + \eta_c \alpha_c + \theta_c \left(\frac{I_c}{I_c^{(0)}} - 1 \right) \right) \quad (42)$$

- N_c 为调整后游客量
- $N_c^{(0)}$ 为基础游客量

- η_c 为价格弹性系数
- θ_c 为投入弹性系数
- $I_c^{(0)}$ 为城市 c 现有投入

(2) 收入模型

门票收入：

$$R_c = N_c \cdot P_c^{(0)}(1 + \alpha_c) \times 10^{-2} \quad (43)$$

- $P_c^{(0)}$ 为基础门票价格
- N_c 为游客数量

衍生收入：

$$S_c = R_c \left(\frac{1}{\beta_c} - 1 \right) \cdot \left(1 + \gamma_c \frac{\Delta D_c}{D_c^{(0)} + \Delta D_c} \right) \quad (44)$$

- β_c 为门票收入占比
- γ_c 为花期弹性系数
- $D_c^{(0)}$ 为基础花期天数
- ΔD_c 为延长花期天数

(3) 经济效益函数

$$\text{效益}_c = R_c + S_c - I_c \quad (45)$$

其中，总目标函数转化为非线性规划问题：

$$\max \sum_{c=1}^4 \left[R_c + R_c \left(\frac{1}{\beta_c} - 1 \right) \left(1 + \gamma_c \frac{\Delta D_c}{D_c^{(0)} + \Delta D_c} \right) - I_c \right] \quad (46)$$

- $\sum_{c=1}^4 I_c \leq 100$
- $\alpha_c \in [-0.3, 0.5]$
- $\Delta D_c \leq \Delta D_c^{\max}$
- $I_c \in [I_c^{\min}, I_c^{\max}]$

根据相关资料的分析,收集到模型的数据,假设数据如下表:

表 19 历史数据表 1

城市	基础游客数量 (万人次)	基础门票价格 (元)	当前花期 (天)	基础设施投入 (亿元)	门票收入 占比
西安	1873	85	88	12.5	0.18
吐鲁番	680	60	88	5.2	0.15
婺源	2370	120	194	18.7	0.12
杭州	3540	150	82	32.8	0.2
西安	1873	85	88	12.5	0.18

表 20 历史数据表 2

城市	价格弹性系数	花期弹性系数	游客弹性系数	价格调整限制	游客容量上限 (万人次)
西安	-0.38	0.55	0.82	0.25	2200
吐鲁番	-0.32	0.68	0.85	0.3	900
婺源	-0.45	0.48	0.75	-0.2	2800
杭州	-0.28	0.62	0.88	0.2	4000
西安	-0.38	0.55	0.82	0.25	2200

注：数据来源：《中国旅游景区投资与管理报告 2024》、《花期调控技术对花卉产业的影响》

6.4.4 模型输出与策略分析

(1) 模型结果的输出

算法选择采用 SLSQP 算法（序列最小二乘规划），适用于带约束的非线性优化问题，能够处理不等式约束（如总投入≤100 亿元）和边界约束（如价格调整范围）。算法通过迭代优化决策变量，最小化目标函数的负值（因 Python 优化库多为最小化设计）

最终，模型推算得到了景区经济效益相关的优化模型的优化结果, 结果如下表：

表 21 数据结果表 1

地区	花卉	门票价格调整率	新门票价格（元）	花期延长（天）	新花期（天）	政府投入（亿元）
西安	杏花	25.00%	106.25	15.0	103.0	18.45
吐鲁番	杏花	30.00%	78.0	15.0	103.0	12.0
婺源	油菜花	10.00%	132.0	20.0	214.0	30.0
杭州	杜鹃花	20.00%	80.0	18.0	100.0	7.5
西安	杏花	25.00%	106.25	15.0	103.0	18.45

表 22 数据结果表 2

地区	花卉	游客数量（万人次）	门票收入（亿元）	总收入（亿元）	经济效益（亿元）
西安	杏花	2425.91	25.78	54.67	136.22
吐鲁番	杏花	1127.28	8.79	64.42	52.42
婺源	油菜花	3209.23	42.36	68.85	338.85
杭州	杜鹃花	3788.15	68.19	378.98	341.48

(2) 关键策略解读

西安:西安依托历史文化与杏花资源，门票提价 25% 至 106.25 元，优化游客结构；利用生物计算延长花期 15 天至 103 天，扩大游客吸引周期。政府投入 18.45 亿元，构建 “文化 + 花卉” 产品体系，吸引 2425.91 万人次游客，实现旅游总收入 154.67 亿元，经济效益显著。

吐鲁番：吐鲁番由于杏花资源稀缺性，门票提价 30%至 78 元，平衡开发与保护成本。延长花期 15 天。政府投入 12 亿元，创新宣传、丰富业态。

婺源：利用油菜花高花期弹性，延长花期 20 天，门票价格上调 10%，通过“品质提升+价格杠杆”最大化收益。

杭州：杭州凭借成熟旅游市场与杜鹃花资源，门票提价 20%至 180 元，优化游客结构。延长花期 18 天至 100 天，丰富产品供给。政府投入 37.5 亿元，推进设施智慧化与服务标准化，吸引 3788.15 万人次游客，旅游总收入 378.98 亿元，推动产业高质量发展。

6.4.5 赏花经济产业链报告

随着人们生活水平的提高和旅游消费的升级，赏花经济逐渐成为城市旅游经济的重要组成部分。西安、吐鲁番、婺源、杭州四城市凭借独特的花卉资源，吸引了大量游客，但在发展过程中也面临诸多挑战。

当前，城市赏花经济存在运营粗放的问题。门票价格缺乏科学调整机制，未能充分考虑市场需求和价格弹性；花期有限，难以持续吸引游客；政府投入分配不合理，未实现资源的高效利用。这些问题导致赏花经济的综合效益未达最优，制约了产业的可持续发展。

为了促进“赏花经”产业链的健康发展出以下建议：

（1）精准价格调控

依据价格弹性理论，对四城市门票价格进行精细化调整。西安依托深厚的历史文化底蕴和杏花资源，提价 25% 优化游客结构；吐鲁番因杏花资源稀缺，提价 30% 平衡开发保护成本；婺源利用油菜花高花期弹性，适度上调 10%；杭州凭借成熟旅游市场和杜鹃花资源，提价 20% 提升收益，通过价格杠杆实现收益最大化。

（2）延长花期策略

运用花期调控技术，延长花卉观赏期。西安、吐鲁番分别将杏花花期延长 15 天，婺源将油菜花花期延长 20 天，杭州将杜鹃花花期延长 18 天。延长花期不仅能增加游客停留时间，还能带动餐饮、住宿、购物等衍生消费，提升整体经济效益。

（3） 优化政府投入

政府投入需在预算约束下实现精准配置。西安投入 18.45 亿元构建“文化+花卉”产品体系；吐鲁番投入 12 亿元创新宣传、丰富业态；婺源投入 30 亿元进行品质提升；杭州投入 37.5 亿元推进设施智慧化与服务标准化，通过合理的投入提升景区吸引力和服务质量。

（4） 产业链协同发展

加强赏花经济产业链上下游协同，整合花卉种植、旅游服务、文创产品开发等环节。开发特色花卉主题文创产品，打造花卉主题餐饮、住宿，举办花卉文化节事活动，提升游客体验，延长游客消费链。

未来发展中，相关政府可以通过构建景区经济效益相关的优化模型，对城市门票价格、花期延长、政府投入进行协同优化，促进赏花经济总效益的提升。同时各城市应进一步延长“赏花经济”产业链，推动花卉种植、旅游服务、文创开发等环节深度融合，打造特色花卉主题餐饮、住宿和文创产品，举办多元化节事活动，提升游客消费体验。同时，持续关注市场动态与游客需求，定期优化模型参数，运用花期调控技术使“赏花经济”拥有“超长花期”，增强对游客的持续吸引力。此外，加强区域间资源共享与经验交流，形成协同发展合力，以赏花经济为驱动力，激活城市经济发展新动能，通过深化产业融合、优化业态布局，推动旅游产业向高质量、可持续方向转型升级，激发赏花经济产业链的发展活力与创新潜力。

6.5 模型的评价、改进与推广

6.5.1 模型的优点

(1) LSTM 机器学习的使用:LSTM 作为特殊的循环神经网络，具有诸多显著优点。它处理序列数据能力出众，既能捕捉长期依赖关系，又能适应不同序列长度，还可自

动提取特征；通过独特门控机制有效避免梯度消失和爆炸问题，实现深层次训练；在各类序列数据预测任务中，其预测准确性高，且对噪声和异常值有一定鲁棒性，稳定性与可靠性强。

(2) 花卉花期预测的有效积温线性回归模型:在物候学理论框架下构建的有效积温模型，具有坚实的生物学与物理学理论基础，能够科学阐释植物生长发育进程与热量积累的内在联系。本文创新性地将有效积温模型与线性回归分析方法相结合，基于最小二乘法对观测数据进行优化拟合。通过构建线性回归模型，系统量化了温度、积温等关键气象因子对油菜花花期的影响规律，以直观的线性关系揭示各气象要素与花期变化的响应机制，为花期预测研究提供了理论支撑与方法创新。

(3) 线性模型与机器学习的混合使用:在预测花卉花期时，. 花卉花期的影响因素复杂多样，可能既存在与花期呈线性关系的因素，也存在一些因素与花期呈现非线性关系。;利用随机森林回归模型处理非线性数据。通过混合使用这两种模型，可以分别对线性和非线性部分进行有效的建模和预测，从而提高整体的预测准确性

(4) 景区经济效益相关的优化模型:考虑了多个城市经济效益密切相关的因素，并且能够体现出这些因素之间的相互关联和相互影响。如模型中体现了门票价格对游客数量的负向影响，以及延长花期通过增加游客数量和门票收入对总利润的正向影响等。这种对因素关联性的考虑，使模型更贴近景区实际运营情况，能够更全面地反映景区经济效益的动态变化。

6.5.2 模型的缺点

(1) 花期预测模型:随机森林回归模型对噪声数据敏感,如果训练数据中存在噪声或异常值，随机森林可能会过度拟合这些数据，导致模型的泛化能力下降。

(2) 景区经济效益相关的优化模型:仅考虑了有限的几个影响因素，而忽略了其他许多可能对经济效益产生影响的因素，如运营成本、市场竞争、季节因素、政策法规变化、游客的消费结构差异等。这些因素在实际情况中可能对经济效益有着重要的影响，忽略它们可能导致模型的准确性和实用性受限

6.5.3 模型的改进

(1) 数据的收集:考虑数据的时效性和代表性,以确保收集到的数据具有时效性,能够反映当前的实际情况。同时,要保证数据具有代表性,能够准确地反映研究对象的特征和规律。

(2) 模型参数的修正:鉴于实际应用场景存在高度复杂性与不确定性,需建立动态反馈机制,基于实时监测数据、业务需求变化及环境变量波动,对模型参数进行系统性校准与优化,以确保模型输出结果与现实情况的高契合度和强适应性。

6.5.4 模型的推广

(1) 随机森林回归模型的推广:在实际应用中,可以通过更精细的特征工程来提升随机森林的性能。在各种数据分析场景中,随机森林回归模型可以用来评估不同特征对目标变量的重要性。通过计算每个特征在模型中的分裂次数、节点纯度提升等指标,确定哪些特征对结果的影响更大。这有助于数据分析师和领域专家更好地理解数据之间的关系,筛选出关键特征,减少数据维度,提高模型的效率和可解释性。

(2) 景区经济效益优化模型的推广:通过数据驱动的决策支持,能够有效提升景区管理水平和政府投资效率。该优化模型可以从单一的赏花经济扩展到整个旅游行业,为景区管理、地方政府决策和旅游投资提供科学依据,实现经济效益与社会效益的最大化。

参考文献

- [1] 黄超, 李巧萍, 谢益军等. 机器学习方法在湖南夏季降雨预测中的应用[J]. 大气科学学报, 2022, 45(2): 191-202 .
- [2] 沈皓俊, 罗勇, 赵宗慈等. 基于 LSTM 网络的中国夏季降雨预测研究[J]. 气候变化研究进展, 2020, 16(3): 263.
- [3] 郭云飞, 等. 高斯函数拟合的地磁日变规律分析[J]. 云南大学学报（自然科学版）, 2024, 46(1): 59 - 66.
- [4] 杨涛, 等. 清镇市樱花始花期预测分析[J]. 南方农业, 2021, 15(31): 183 - 187.
- [5] 王艳晓. 近 50 年气候变化对洛阳牡丹花期的影响[J]. 农业灾害研究, 2022, 12(8): 88 - 90.
- [6] 李德, 陈文涛, 乐章燕, 等. 基于随机森林算法和气象因子的砀山酥梨始花期预报[J]. 农业工程学报, 2020, 36(12): 143-151 .
- [7] 沙世琨. 基于随机森林算法的陈垓灌区降水量预测模型[J]. 水利技术监督, 2020, (05): 134-137 .

附录

问题一源程序代码：

```
import pandas as pd

import numpy as np

import torch

import torch.nn as nn

from torch.utils.data import Dataset, DataLoader

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

import re


# 参数配置

LOOKBACK = 3 # 使用 3 年历史数据作为输入

PRED_STEPS = 6 # 预测未来 6 天的数据（2025 年和 2026 年各 3 天）

FEATURES = ['降水量(mm)', '持续时间(小时)']

EPOCHS = 1000

BATCH_SIZE = 8


# 改进的日期预处理函数

def preprocess_date(date_str):

    # 提取数字部分并格式化为"月-日"

    parts = re.findall(r'\d+', str(date_str))

    if len(parts) >= 2:

        month = parts[0].lstrip('0') # 移除前导零

        day = parts[1].lstrip('0')

        return f'{month}-{day}'

    return np.nan
```



```
# 数据预处理

def prepare_data(df):

# 深拷贝原始数据

df = df.copy()


# 预处理日期列

df['处理后的日期'] = df['日期'].apply(preprocess_date)


# 生成完整日期列

df['完整日期'] = df['年份'].astype(str) + '-' + df['处理后的日期']


# 转换日期格式

df['日期'] = pd.to_datetime(df['完整日期'], format='%Y-%m-%d', errors='coerce')


# 处理无效日期

invalid_mask = df['日期'].isnull()

if invalid_mask.any():

print(f'发现 {invalid_mask.sum()} 条无效日期记录：')

print(df[invalid_mask][['年份', '日期', '处理后的日期']])

df = df[~invalid_mask].copy()


# 检查剩余数据量

if len(df) < (LOOKBACK + 1) * 3: # 至少需要 4 年数据（3 年输入+1 年预测）

raise ValueError(f'有效数据不足！至少需要 {(LOOKBACK + 1) * 3} 条数据，当前只有 {len(df)} 条')
```

```
df = df.sort_values('日期').reset_index(drop=True)

# 数据标准化
if len(df) == 0:
    raise ValueError("标准化前数据为空，请检查日期处理逻辑！")

scaler = StandardScaler()
df[FEATURES] = scaler.fit_transform(df[FEATURES])

# 构建时间序列数据集
X, y_reg, y_cls = [], [], []
total_required = LOOKBACK * 3 + PRED_STEPS
for i in range(len(df) - total_required + 1):
    input_seq = df.iloc[i:i + LOOKBACK * 3][FEATURES].values
    output_seq = df.iloc[i + LOOKBACK * 3:i + LOOKBACK * 3 +
PRED_STEPS][FEATURES].values

    cls_labels = []
    for row in output_seq:
        precip = row[0] * scaler.scale_[0] + scaler.mean_[0]
        duration = row[1] * scaler.scale_[1] + scaler.mean_[1]
        # 修正负降雨量为 0（训练数据标签生成时处理）
        if precip < 0:
            precip = 0
        cls_labels.append(1 if 0.1 <= precip <= 10.0 and duration >= 3 else 0)

    X.append(input_seq)
    y_reg.append(output_seq.flatten())
```

```
y_cls.append(cls_labels)
```

```
if len(X) == 0:
```

```
    raise ValueError("无法生成训练样本，请检查：\n"
```

```
    f"1. 数据量是否足够（至少需要{total_required}条连续数据）\n"
```

```
    "2. 日期是否连续无缺失\n"
```

```
    "3. 特征列是否存在空值")
```

```
return np.array(X), np.array(y_reg), np.array(y_cls), scaler
```

```
class WeatherDataset(Dataset):
```

```
    def __init__(self, X, y_reg, y_cls):
```

```
        self.X = torch.FloatTensor(X)
```

```
        self.y_reg = torch.FloatTensor(y_reg)
```

```
        self.y_cls = torch.FloatTensor(y_cls)
```

```
    def __len__(self):
```

```
        return len(self.X)
```

```
    def __getitem__(self, idx):
```

```
        return self.X[idx], (self.y_reg[idx], self.y_cls[idx])
```

```
class DualLSTM(nn.Module):
```

```
    def __init__(self, input_size, hidden_size=64):
```

```
        super().__init__()
```

```
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
```

```
self.reg_head = nn.Sequential(
    nn.Linear(hidden_size, 32),
    nn.ReLU(),
    nn.Linear(32, PRED_STEPS * 2)
)

self.cls_head = nn.Sequential(
    nn.Linear(hidden_size, 32),
    nn.ReLU(),
    nn.Linear(32, PRED_STEPS)
)

def forward(self, x):
    lstm_out, _ = self.lstm(x)
    last_out = lstm_out[:, -1, :]
    reg = self.reg_head(last_out)
    cls = torch.sigmoid(self.cls_head(last_out))
    return reg.view(-1, PRED_STEPS, 2), cls

def train(model, dataloader, optimizer):
    model.train()
    total_loss = 0
    for X, (y_reg, y_cls) in dataloader:
        optimizer.zero_grad()
        pred_reg, pred_cls = model(X)

        reg_loss = nn.MSELoss()(pred_reg.view(-1, PRED_STEPS * 2), y_reg)
        cls_loss = nn.BCELoss()(pred_cls, y_cls)
```

```
loss = reg_loss + cls_loss
```

```
loss.backward()
```

```
optimizer.step()
```

```
total_loss += loss.item()
```

```
return total_loss / len(data_loader)
```

```
# 评估模型性能（修改召回率计算逻辑）
```

```
def evaluate_model(pred_reg, actual_data, scaler):
```

```
# 反标准化预测值
```

```
pred_reg = scaler.inverse_transform(pred_reg)
```

```
# 修正预测中的负值
```

```
for i in range(len(pred_reg)):
```

```
    if pred_reg[i, 0] < 0:
```

```
        pred_reg[i, 0] = 0
```

```
    if pred_reg[i, 1] < 0:
```

```
        pred_reg[i, 1] = 0
```

```
# 提取实际数据
```

```
actual_precip = actual_data['降水量(mm)'].values
```

```
actual_duration = actual_data['持续时间(小时)'].values
```

```
# 计算预测和实际的雨纷纷状态
```

```
pred_rainy = [(0.1 <= p <= 10.0 and d >= 3) for p, d in pred_reg]
```

```
actual_rainy = [(0.1 <= p <= 10.0 and d >= 3) for p, d in zip(actual_precip, actual_duration)]
```

```
# 计算混淆矩阵元素

tp = sum([p and a for p, a in zip(pred_rainy, actual_rainy)])
fp = sum([p and not a for p, a in zip(pred_rainy, actual_rainy)])
fn = sum([not p and a for p, a in zip(pred_rainy, actual_rainy)])
tn = sum([not p and not a for p, a in zip(pred_rainy, actual_rainy)])

# 计算准确率和召回率（关键修改：无正例时召回率设为 1）

total = tp + fp + fn + tn
accuracy = (tp + tn) / total if total > 0 else 0.0
if (tp + fn) == 0: # 实际无正例（全为 False）时，召回率定义为 1
    recall = 1.0
else:
    recall = tp / (tp + fn)

return {
    'tp': tp, 'fp': fp, 'fn': fn, 'tn': tn,
    'accuracy': accuracy, 'recall': recall,
    'pred_rainy': pred_rainy, 'actual_rainy': actual_rainy
}

if __name__ == "__main__":
    try:
        # 读取训练数据
        df = pd.read_excel('代码数据.xlsx', sheet_name='杭州')

        # 数据预处理
        X, y_reg, y_cls, scaler = prepare_data(df)
```

划分数据集

```
X_train, X_test, y_reg_train, y_reg_test, y_cls_train, y_cls_test = train_test_split(
    X, y_reg, y_cls, test_size=0.2, shuffle=False)
```

检查训练数据

```
if len(X_train) == 0:
    raise ValueError("训练数据为空，请增加数据量或调整 LOOKBACK 参数")
```

创建数据加载器

```
train_dataset = WeatherDataset(X_train, y_reg_train, y_cls_train)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
```

初始化模型

```
model = DualLSTM(input_size=len(FEATURES))
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

训练

```
for epoch in range(EPOCHS):
    loss = train(model, train_loader, optimizer)
    if (epoch + 1) % 20 == 0:
        print(f'Epoch {epoch + 1}/{EPOCHS}, Loss: {loss:.4f}')
```

预测

```
with torch.no_grad():
    last_input = torch.FloatTensor(X[-1:])
    pred_reg, pred_cls = model(last_input)
```

```
# 反标准化

pred_reg = pred_reg.numpy().reshape(-1, 2)

pred_reg = scaler.inverse_transform(pred_reg)


# 修正预测中的负值（同时处理降水量和持续时间）

for i in range(len(pred_reg)):

    # 降水量负值修正

    if pred_reg[i, 0] < 0:

        pred_reg[i, 0] = 0

    # 持续时间负值修正

    if pred_reg[i, 1] < 0:

        pred_reg[i, 1] = 0


# 结果展示

def format_results(data, year):

    results = []

    daily_conditions = [] # 新增：记录每天是否雨纷纷

    for i, (precip, duration) in enumerate(data):

        day = 4 + i

        condition = "是" if 0.1 <= precip <= 10.0 and duration >= 3 else "否"

        daily_conditions.append(condition == "是") # 转换为布尔值

        results.append(

            f'{year}年4月{day}日 | 降水量: {precip:.2f}mm | 持续时间: {duration:.2f}h | 雨纷纷:

            {condition}')

    return results, daily_conditions # 返回结果列表和布尔列表
```



```
# 处理 2025 年和 2026 年的预测结果

pred_2025, cond_2025 = format_results(pred_reg[:3], 2025)
pred_2026, cond_2026 = format_results(pred_reg[3:6], 2026)

print("\n 预测结果: ")
print("\n".join(pred_2025))
if any(cond_2025):
    print(f"\n2025 年清明假期（4 月 4-6 日）期间出现雨纷纷天气")
else:
    print(f"\n2025 年清明假期（4 月 4-6 日）期间不出现雨纷纷天气")

print("\n" + "-" * 50 + "\n") # 分隔线
print("\n".join(pred_2026))
if any(cond_2026):
    print(f"\n2026 年清明假期（4 月 4-6 日）期间出现雨纷纷天气")
else:
    print(f"\n2026 年清明假期（4 月 4-6 日）期间不出现雨纷纷天气")

# 读取 2025 年实际数据

try:
    df_actual_2025 = pd.read_excel("2025 的数据.xlsx", sheet_name='杭州')
    print("\n 成功读取 2025 年实际数据...")

# 确保实际数据包含所需的列

required_columns = ['年份', '日期', '降水量(mm)', '持续时间(小时)']
for col in required_columns:
    if col not in df_actual_2025.columns:
        raise ValueError(f"实际数据中缺少必要的列: {col}")
```

```
# 筛选 2025 年 4 月 4-6 日的数据
df_actual_2025['日期'] = df_actual_2025['日期'].astype(str)
df_actual_2025['处理后的日期'] = df_actual_2025['日期'].apply(preprocess_date)
df_actual_2025['完整日期'] = df_actual_2025['年份'].astype(str) + '-' + df_actual_2025['处理后的日期']

df_actual_2025['日期'] = pd.to_datetime(df_actual_2025['完整日期'], format='%Y-%m-%d')

# 筛选目标日期范围
start_date = pd.to_datetime('2025-04-04')
end_date = pd.to_datetime('2025-04-06')
mask = (df_actual_2025['日期'] >= start_date) & (df_actual_2025['日期'] <= end_date)
actual_data_2025 = df_actual_2025.loc[mask].sort_values('日期')

if len(actual_data_2025) != 3:
    print(f'警告：2025 年清明假期数据不完整，应有 3 天，实际有 {len(actual_data_2025)} 天')

# 评估模型性能
evaluation = evaluate_model(pred_reg[:3], actual_data_2025, scaler)

# 输出评估结果
print("\n" + "-" * 50 + "\n")
print("模型评估结果：")
print(f'准确率: {evaluation['accuracy']:.2%}')
print(f'召回率: {evaluation['recall']:.2%}')
print("\n 详细对比:")
for i, (pred, actual, date) in enumerate(zip(
    evaluation['pred_rainy'],
```

```

evaluation['actual_rainy'],

actual_data_2025['日期'].dt.strftime('%Y 年%m 月%d 日')

)):

print(f'{date}: 预测雨纷纷={pred}, 实际雨纷纷={actual}')

except Exception as e:

print(f'\n 读取 2025 年实际数据时出错: {str(e)}')

print("请确保文件存在且格式正确（包含年份、日期、降水量、持续时间列）")

except Exception as e:

print(f'\n 程序运行出错: {str(e)}')

print("可能的原因及解决方案：")

print("1. 日期格式问题 → 确保日期列格式为'4-4'或'4 月 4 日'")

print("2. 数据量不足 → 至少需要 4 年完整数据（当前 LOOKBACK=3）")

print("3. 特征列不匹配 → 检查 Excel 列名是否与代码中 FEATURES 定义一致")

print("4. 无效日期 → 查看打印的无效日期记录并修正原始数据")

```

问题二源程序代码：

```

# -*- coding: utf-8 -*-

"""

Created on Mon May 12 03:48:47 2025

@author: l'w

"""

import pandas as pd

from sklearn.ensemble import RandomForestRegressor

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```

```
# 定义预测函数，支持不同花卉数据并输出模型检验结果

def predict_flower_data(file_path, flower_name):

# 加载数据（直接使用相对路径，同一文件夹下直接写文件名）

df = pd.read_excel(file_path)

# 准备特征和目标变量（保留原逻辑不变）

X = df[['积温（GDD）', '日均温（℃）', '日照时长（小时）', '降水量（mm）', '土壤湿度（%）']]

y_start = df['始期_当年第几天']

y_duration = df['花期（天）']

# 分割数据集为训练集和测试集（保留原逻辑不变）

X_train, X_test, y_start_train, y_start_test = train_test_split(
X, y_start, test_size=0.2, random_state=42)

_, _, y_duration_train, y_duration_test = train_test_split(
X, y_duration, test_size=0.2, random_state=42)

# 始期模型和花期模型（保留原逻辑不变）

rf_start = RandomForestRegressor(n_estimators=100, random_state=42)

rf_start.fit(X_train, y_start_train)

rf_duration = RandomForestRegressor(n_estimators=100, random_state=42)

rf_duration.fit(X_train, y_duration_train)

# 交叉验证（保留原逻辑不变）

cv_start = cross_val_score(rf_start, X, y_start, cv=3,
scoring='neg_mean_absolute_error')
```

```
cv_duration = cross_val_score(rf_duration, X, y_duration, cv=3,
scoring='neg_mean_absolute_error')

# 预测和评估（保留原逻辑不变）
y_start_pred = rf_start.predict(X_test)
y_duration_pred = rf_duration.predict(X_test)

metrics_start = {
    'MAE': mean_absolute_error(y_start_test, y_start_pred),
    'RMSE': mean_squared_error(y_start_test, y_start_pred, squared=False),
    'R²': r2_score(y_start_test, y_start_pred),
    '交叉验证 MAE': -cv_start.mean()
}

metrics_duration = {
    'MAE': mean_absolute_error(y_duration_test, y_duration_pred),
    'RMSE': mean_squared_error(y_duration_test, y_duration_pred, squared=False),
    'R²': r2_score(y_duration_test, y_duration_pred),
    '交叉验证 MAE': -cv_duration.mean()
}

# 生成 2026 年假设数据（基于文件最后一条数据，保留原逻辑不变）
last_year = df.iloc[-1]
avg_gdd_increase = df['积温（GDD）'].diff().mean()

hypo_2026 = {
    '积温（GDD）': last_year['积温（GDD）'] + avg_gdd_increase,
    '日均温（℃）': last_year['日均温（℃）'] + 0.1,
```

```
'日照时长（小时）': last_year['日照时长（小时）'] + 2,
'降水量（mm）': last_year['降水量（mm）'],
'土壤湿度（%）': last_year['土壤湿度（%）'] + 1
}

hypo_2026_df = pd.DataFrame([hypo_2026])
start_2026_rf = rf_start.predict(hypo_2026_df)[0]
duration_2026_rf = rf_duration.predict(hypo_2026_df)[0]

# 输出结果（保留原逻辑不变）
print(f"{flower_name}花期预测模型评估结果：")
print("\n 始期预测模型：")
for key, value in metrics_start.items():
    print(f"{key}: {value:.2f}")

print("\n 花期预测模型：")
for key, value in metrics_duration.items():
    print(f"{key}: {value:.2f}")

print(f"\n{flower_name}2026 年预测结果：")
print(f"始期: {start_2026_rf:.1f}天（当年第几天）")
print(f"花期: {duration_2026_rf:.1f}天")

return {
    'start_model_metrics': metrics_start,
    'duration_model_metrics': metrics_duration,
    'prediction_2026': {
        'start_day': start_2026_rf,
```

```
'duration_days': duration_2026_rf
}

}

# =====
# 调用示例（处理两个表格）
# =====
if __name__ == "__main__":
    # 处理杜鹃花数据
    results_azalea = predict_flower_data(
        file_path="杜鹃花.xlsx", # 直接使用文件名（同一文件夹）
        flower_name="杜鹃花"
    )

    # 处理杏花数据（新增调用）
    results_apricot = predict_flower_data(
        file_path="杏花.xlsx", # 直接使用文件名（同一文件夹）
        flower_name="杏花"
    )
```

问题三源程序代码：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from math import radians, sin, cos, sqrt, asin

# 设置图片清晰度
```

```
plt.rcParams['figure.dpi'] = 300

# 正常显示中文
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']

# 定义计算球面距离的函数
def haversine(lat1, lon1, lat2, lon2):
    # 将十进制度数转化为弧度
    lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2])

    # Haversine 公式
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = sin(dlat / 2) ** 2 + cos(lat1) * cos(lat2) * sin(dlon / 2) ** 2
    c = 2 * asin(sqrt(a))
    r = 6371 # 地球平均半径，单位为公里
    return c * r

# 读取 Excel 文件
excel_file = pd.ExcelFile(r'2026 年问题三.xlsx')

# 读取数据
df_sun_temp_flower = excel_file.parse('26 年日照、均温与花期')
df_rain = excel_file.parse('26 年降雨')
df_location = excel_file.parse('经纬度')
df_speed = excel_file.parse('高铁速度')
```



```
# 提取高铁速度
high_speed_rail_speed = df_speed['时速 (km/h) '].values[0]

# 定义计算天气评分的函数
def weather_score(rainfall, temperature, sunshine):
    # 降雨评分
    if rainfall <= 10 and (df_rain[['2026.4.4', '2026.4.5', '2026.4.6']] <=
3).all().all():
        rain_score = 100
    elif 10 < rainfall <= 25 and (3 < df_rain[['2026.4.4', '2026.4.5', '2026.4.6']]
<= 6).all().all():
        rain_score = 60
    else:
        rain_score = 30

    # 温度评分
    temp_score = 100 * np.exp(-((temperature - 20) / 5) ** 2)

    # 光照评分
    sun_score = (sunshine - 4) / (12 - 4) * 100 if 4 <= sunshine <= 12 else 0 if sunshine
< 4 else 100

    # 综合天气评分
    return 0.4 * rain_score + 0.3 * temp_score + 0.3 * sun_score
```

```
# 定义计算距离评分的函数

def distance_score(distance):

    min_distance = 100

    max_distance = 1000

    return 100 - (distance - min_distance) / (max_distance - min_distance) * 60


# 定义计算交通出行时间评分的函数

def travel_time_score(travel_time):

    min_time = 1

    max_time = 6

    return 100 - (travel_time - min_time) / (max_time - min_time) * 80


# 定义计算假期花期重叠度评分的函数

def flower_overlap_score(start_day, flower_days):

    end_day = start_day + flower_days - 1

    qingming_start = 94 # 2026 年 4 月 4 日是第 94 天

    qingming_end = 96 # 2026 年 4 月 6 日是第 96 天

    overlap_days = max(0, min(end_day, qingming_end) - max(start_day, qingming_start)

+ 1)

    if overlap_days == 0:

        return 0

    elif overlap_days == 1:

        return 40

    elif overlap_days == 2:

        return 70
```

```
else:

    return 100


# 计算华东路线（婺源 - 杭州）的各项评分

# 天气评分

east_weather_score = weather_score(

    df_rain[df_rain['城市降水量 (mm)'].isin(['婺源', '杭州'])][['2026.4.4',
'2026.4.5', '2026.4.6']].sum().sum(),

    df_sun_temp_flower[df_sun_temp_flower['气象因子'].isin(['杏花（华东）', '杜鹃花
（华东）'])][['日均温 (°C)']].mean(),

    df_sun_temp_flower[df_sun_temp_flower['气象因子'].isin(['杏花（华东）', '杜鹃花
（华东）'])][

        '26 年清明假期日照预测'].mean())

# 距离评分

east_distance = haversine(float(df_location[df_location['城市'] == '婺源']['纬
度'].values[0].strip('°N')),

    float(df_location[df_location['城市'] == '婺源']['经度
'].values[0].strip('°E')),

    float(df_location[df_location['城市'] == '杭州']['纬度
'].values[0].strip('°N')),

    float(df_location[df_location['城市'] == '杭州']['经度
'].values[0].strip('°E')))

east_distance_score = distance_score(east_distance)

# 交通出行时间评分

east_travel_time = east_distance / high_speed_rail_speed
```

```

east_travel_time_score = travel_time_score(east_travel_time)

# 假期花期重叠度评分
east_flower_overlap_score = flower_overlap_score(
    df_sun_temp_flower[df_sun_temp_flower['气象因子'].isin(['杏花（华东）', '杜鹃花
（华东）'])][
    '始花期（26年的第几天）（天）'].min(),
    df_sun_temp_flower[df_sun_temp_flower['气象因子'].isin(['杏花（华东）', '杜鹃花
（华东）'])]['花期天数（天）'].sum())

# 计算西北地区路线（西安 - 吐鲁番）的各项评分
# 天气评分
northwest_weather_score = weather_score(
    df_rain[df_rain['城市降水量（mm）'].isin(['西安', '吐鲁番'])][['2026.4.4',
'2026.4.5', '2026.4.6']].sum().sum(),
    df_sun_temp_flower[df_sun_temp_flower['气象因子'] == '油菜花（西北）']['日均温（℃）
'].values[0],
    df_sun_temp_flower[df_sun_temp_flower['气象因子'] == '油菜花（西北）']['26年清
明假期日照预测'].values[0])

# 距离评分
northwest_distance = haversine(float(df_location[df_location['城市'] == '西安
']['纬度'].values[0].strip('°N')),
    float(df_location[df_location['城市'] == '西安']['经度
'].values[0].strip('°E')),
    float(df_location[df_location['城市'] == '吐鲁番']['纬度
'].values[0].strip('°N')),

```

```
float(df_location[df_location['城市'] == '吐鲁番']['经度
'].values[0].strip('°E'))

northwest_distance_score = distance_score(northwest_distance)

# 交通出行时间评分

northwest_travel_time = northwest_distance / high_speed_rail_speed

northwest_travel_time_score = travel_time_score(northwest_travel_time)

# 假期花期重叠度评分

northwest_flower_overlap_score = flower_overlap_score(
df_sun_temp_flower[df_sun_temp_flower['气象因子'] == '油菜花（西北）']['始花期（26
年的第几天）（天）'].values[0],
df_sun_temp_flower[df_sun_temp_flower['气象因子'] == '油菜花（西北）']['花期天数
（天）'].values[0])

# 创建评分数据 DataFrame

scores = pd.DataFrame({
'路线': ['华东路线', '西北地区路线'],
'天气评分': [east_weather_score, northwest_weather_score],
'距离评分': [east_distance_score, northwest_distance_score],
'交通出行时间评分': [east_travel_time_score, northwest_travel_time_score],
'假期花期重叠度评分': [east_flower_overlap_score,
northwest_flower_overlap_score]
})

# 定义层次分析法(AHP)计算权重

def ahp_weight(comparison_matrix):
```

```
"""使用层次分析法计算权重"""

# 计算特征值和特征向量

eigenvalues, eigenvectors = np.linalg.eig(comparison_matrix)

# 获取最大特征值对应的特征向量

max_index = np.argmax(eigenvalues)

eigenvector = eigenvectors[:, max_index].real

# 归一化处理

weights = eigenvector / eigenvector.sum()

# 一致性检验

n = comparison_matrix.shape[0]

CI = (np.max(eigenvalues).real - n) / (n - 1)

# 随机一致性指标(RI)查表

RI_dict = {1: 0, 2: 0, 3: 0.58, 4: 0.90, 5: 1.12, 6: 1.24, 7: 1.32, 8: 1.41, 9:
1.45}

RI = RI_dict.get(n, 0)

# 一致性比率

CR = CI / RI if RI != 0 else 0

return weights, CR

# 构建判断矩阵（示例矩阵，可根据实际情况调整）

comparison_matrix = np.array([
```

[1, 2, 3, 1], # 天气评分比距离评分稍微重要(2)，比交通时间明显重要(3)，与花期重叠度同等重要(1)

[1 / 2, 1, 2, 1 / 2], # 距离评分比天气评分稍微不重要(1/2)，比交通时间稍微重要(2)，比花期重叠度稍微不重要(1/2)

[1 / 3, 1 / 2, 1, 1 / 3], # 交通时间比天气评分明显不重要(1/3)，比距离评分稍微不重要(1/2)，比花期重叠度明显不重要(1/3)

[1, 2, 3, 1] # 花期重叠度比天气评分同等重要(1)，比距离评分稍微重要(2)，比交通时间明显重要(3)

])

计算权重和一致性比率

weights, CR = ahp_weight(comparison_matrix)

打印权重和一致性检验结果

print("层次分析法权重计算结果:")

print(f"天气评分权重: {weights[0]:.4f}")

print(f"距离评分权重: {weights[1]:.4f}")

print(f"交通出行时间评分权重: {weights[2]:.4f}")

print(f"假期花期重叠度评分权重: {weights[3]:.4f}")

print(f"一致性比率 CR: {CR:.4f}")

if CR < 0.1:

print("一致性检验通过，权重分配合理")

else:

print("一致性检验未通过，建议调整判断矩阵")

计算加权总分

score_columns = ['天气评分', '距离评分', '交通出行时间评分', '假期花期重叠度评分']

```
scores['总分'] = scores[score_columns].dot(weights).round(2)
```

```
print('\n 两条线路的各项评分及 AHP 加权总分: \n', scores)
```

问题四源程序代码:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties
from scipy.optimize import minimize
import seaborn as sns

# 设置中文字体
try:
    font = FontProperties(fname=r"C:\Windows\Fonts\simhei.ttf") # Windows 系统
except:
    try:
        font = FontProperties(fname="/System/Library/Fonts/PingFang.ttc") # macOS 系统
    except:
        font = None # 使用默认字体

class FlowerEconomyOptimizationModelV5:
    def __init__(self):
        # 城市数据
        self.cities = ["西安", "吐鲁番", "婺源", "杭州"]
        self.city_flowers = {
            "西安": "杏花",
```



```
"吐鲁番": "杏花",  
"婺源": "油菜花",  
"杭州": "杜鹃花"  
}
```

基础数据（基于实际情况和公开资料）

```
self.base_data = {  
    "西安": {  
        "基础游客数量(万人次)": 1873, # 2023 年西安赏花季游客量  
        "基础门票价格(元)": 85, # 西安主要赏花景区平均门票  
        "当前花期(天)": 88, # 西安杏花平均花期  
        "基础设施投入(亿元)": 12.5, # 参考西安文旅基础设施投资  
        "门票收入占比": 0.18, # 门票收入占旅游总收入比例  
        "价格弹性系数": -0.38, # 参考旅游经济学文献  
        "花期弹性系数": 0.55, # 花期延长对收入的影响  
        "游客弹性系数": 0.82, # 政府投入对游客数量的影响系数  
        "价格调整限制": 0.25, # 最大提价 25%  
        "游客容量上限": 2200, # 景区最大承载量  
    },  
    "吐鲁番": {  
        "基础游客数量(万人次)": 680, # 2023 年吐鲁番杏花节游客量  
        "基础门票价格(元)": 60, # 吐鲁番杏花景区平均门票  
        "当前花期(天)": 88, # 吐鲁番杏花平均花期  
        "基础设施投入(亿元)": 5.2, # 参考吐鲁番文旅投资数据  
        "门票收入占比": 0.15, # 门票收入占旅游总收入比例  
        "价格弹性系数": -0.32, # 参考旅游经济学文献  
        "花期弹性系数": 0.68, # 花期延长对收入的影响  
        "游客弹性系数": 0.85, # 政府投入对游客数量的影响系数
```

```
"价格调整限制": 0.30, # 最大提价 30%

"游客容量上限": 900, # 景区最大承载量

},

"婺源": {

"基础游客数量(万人次)": 2370, # 2023 年婺源油菜花季游客量

"基础门票价格(元)": 120, # 婺源油菜花景区联票价格

"当前花期(天)": 194, # 婺源油菜花平均花期

"基础设施投入(亿元)": 18.7, # 参考婺源文旅投资数据

"门票收入占比": 0.12, # 门票收入占旅游总收入比例

"价格弹性系数": -0.45, # 参考旅游经济学文献

"花期弹性系数": 0.48, # 花期延长对收入的影响

"游客弹性系数": 0.75, # 政府投入对游客数量的影响系数

"价格调整限制": -0.20, # 最大降价 20%

"游客容量上限": 2800, # 景区最大承载量

},

"杭州": {

"基础游客数量(万人次)": 3540, # 2023 年杭州赏花季游客量

"基础门票价格(元)": 150, # 杭州主要赏花景区平均门票

"当前花期(天)": 82, # 杭州杜鹃花平均花期

"基础设施投入(亿元)": 32.8, # 参考杭州文旅投资数据

"门票收入占比": 0.2, # 门票收入占旅游总收入比例

"价格弹性系数": -0.28, # 参考旅游经济学文献

"花期弹性系数": 0.62, # 花期延长对收入的影响

"游客弹性系数": 0.88, # 政府投入对游客数量的影响系数

"价格调整限制": 0.20, # 最大提价 20%

"游客容量上限": 4000, # 景区最大承载量

}

}
```

```
# 花期延长上限（根据文献和实际情况设定）

self.flower_period_limits = {

    "油菜花": 20, # 最多延长 20 天

    "杏花": 15, # 最多延长 15 天

    "杜鹃花": 18, # 最多延长 18 天

}


# 政府投入区间（根据地方财政能力和行业标准）

self.government_investment_ranges = {

    "西安": (12, 22), # 政府投入区间为 12-22 亿元

    "吐鲁番": (5, 12), # 政府投入区间为 5-12 亿元

    "婺源": (18, 30), # 政府投入区间为 18-30 亿元

    "杭州": (30, 45), # 政府投入区间为 30-45 亿元

}


# 决策变量边界

self.bounds = self._create_decision_bounds()


# 约束条件

self.constraints = self._create_constraints()


def _create_decision_bounds(self):

    """创建决策变量的边界"""

    bounds = []

    for city in self.cities:

        # 门票价格调整率（根据城市特点设置不同限制）
```

```
price_limit = self.base_data[city]["价格调整限制"]

if price_limit > 0: # 提价限制
    bounds.append((-0.15, price_limit)) # 最大降价 15%，最大提价由城市设定
else: # 降价限制
    bounds.append((price_limit, 0.10)) # 最大降价由城市设定，最大提价 10%

# 花期延长时间
flower_type = self.city_flowers[city]
bounds.append((0, self.flower_period_limits[flower_type]))

# 政府投入资金
min_investment, max_investment = self.government_investment_ranges[city]
bounds.append((min_investment, max_investment))

return bounds

def _create_constraints(self):
    """创建约束条件"""
    constraints = []

    # 四个城市分别的资金投入区间约束
    for i, city in enumerate(self.cities):
        min_investment, max_investment = self.government_investment_ranges[city]

    def investment_constraint(x, index=i):
        investment = x[index * 3 + 2]
        return min_investment - investment, investment - max_investment
```

```
constraints.extend([
    {'type': 'ineq', 'fun': lambda x, index=i: investment_constraint(x, index)[0]},
    {'type': 'ineq', 'fun': lambda x, index=i: investment_constraint(x, index)[1]}
])
```

游客数量不超过容量上限

```
def visitor_capacity_constraint(x):
    for i, city in enumerate(self.cities):
        price_adjustment = x[i * 3]
        government_investment = x[i * 3 + 2]
        base_visitors = self.base_data[city]["基础游客数量(万人次)"]
        price_effect = self.base_data[city]["价格弹性系数"] * price_adjustment
        investment_effect = self.base_data[city]["游客弹性系数"] * (
            government_investment / self.base_data[city]["基础设施投入(亿元)"] - 1)
        new_visitors = base_visitors * (1 + price_effect + investment_effect)
        capacity = self.base_data[city]["游客容量上限"]
        if new_visitors > capacity:
            return capacity - new_visitors
    return 1 # 所有城市游客数量都满足条件
```

```
constraints.append({'type': 'ineq', 'fun': visitor_capacity_constraint})
```

确保经济效益为正

```
def positive_benefit_constraint(x):
    for i, city in enumerate(self.cities):
        price_adjustment = x[i * 3]
        period_extension = x[i * 3 + 1]
        government_investment = x[i * 3 + 2]
```

```
base_data = self.base_data[city]

new_ticket_price = base_data["基础门票价格(元)"] * (1 + price_adjustment)

price_effect = base_data["价格弹性系数"] * price_adjustment

investment_effect = base_data["游客弹性系数"] * (
    government_investment / self.base_data[city]["基础设施投入(亿元)"] - 1)

# 添加游客数量增长的非线性因素（避免过度增长）

growth_factor = 1 + price_effect + investment_effect

if growth_factor > 1.3: # 限制最大增长 30%
    growth_factor = 1.3 + 0.5 * (growth_factor - 1.3) # 超过部分增长放缓

new_visitor_count = base_data["基础游客数量(万人次)"] * growth_factor

ticket_revenue = new_visitor_count * new_ticket_price / 10000

new_period = base_data["当前花期(天)"] + period_extension

# 计算总收入（添加非线性因素，避免过度乐观）

base_total_revenue = ticket_revenue / base_data["门票收入占比"]

period_effect = 1 + base_data["花期弹性系数"] * period_extension / new_period

# 考虑规模效应，收入增长随规模增大而减缓

scale_factor = min(1.0, 1.2 - 0.1 * np.log(new_visitor_count / 1000))

total_revenue = base_total_revenue * period_effect * scale_factor

economic_benefit = total_revenue - government_investment

if economic_benefit <= 0:

    return economic_benefit
```

```
return 1 # 所有城市经济效益都为正

constraints.append({'type': 'ineq', 'fun': positive_benefit_constraint})

return constraints

def objective_function(self, x):
    """目标函数：最大化每个城市的经济效益"""
    total_benefit = 0

    for i, city in enumerate(self.cities):
        price_adjustment = x[i * 3]
        period_extension = x[i * 3 + 1]
        government_investment = x[i * 3 + 2]

        base_data = self.base_data[city]

        new_ticket_price = base_data["基础门票价格(元)"] * (1 + price_adjustment)
        price_effect = base_data["价格弹性系数"] * price_adjustment
        investment_effect = base_data["游客弹性系数"] * (
            government_investment / base_data["基础设施投入(亿元)"] - 1)

        # 添加游客数量增长的非线性因素（避免过度增长）
        growth_factor = 1 + price_effect + investment_effect

        if growth_factor > 1.3: # 限制最大增长 30%
            growth_factor = 1.3 + 0.5 * (growth_factor - 1.3) # 超过部分增长放缓

        new_visitor_count = base_data["基础游客数量(万人次)"] * growth_factor
```

```
ticket_revenue = new_visitor_count * new_ticket_price / 10000

new_period = base_data["当前花期(天)"] + period_extension

# 计算总收入（添加非线性因素，避免过度乐观）

base_total_revenue = ticket_revenue / base_data["门票收入占比"]

period_effect = 1 + base_data["花期弹性系数"] * period_extension / new_period

# 考虑规模效应，收入增长随规模增大而减缓

scale_factor = min(1.0, 1.2 - 0.1 * np.log(new_visitor_count / 1000))

total_revenue = base_total_revenue * period_effect * scale_factor

economic_benefit = total_revenue - government_investment

total_benefit += economic_benefit

# 取负值，因为我们要最大化经济效益，而 minimize 函数是求最小值

return -total_benefit

def optimize(self):
    """求解优化问题"""

    # 初始猜测值

    initial_guess = []

    for city in self.cities:
        min_investment, max_investment = self.government_investment_ranges[city]

        # 设置初始猜测值为上下限的中间值

        initial_guess.extend([0, 0, (min_investment + max_investment) / 2])

    # 使用 SLSQP 方法求解约束优化问题
```



```
result = minimize(
    self.objective_function,
    initial_guess,
    method='SLSQP',
    bounds=self.bounds,
    constraints=self.constraints,
    options={'disp': True, 'maxiter': 1000}
)

return result

def format_optimization_results(self, result):
    """格式化优化结果"""
    if not result.success:
        print("优化失败:", result.message)
        # 尝试获取次优解
        print("尝试使用当前解作为次优解...")
        if hasattr(result, 'x'):
            optimal_solution = result.x
            optimal_benefit = -self.objective_function(optimal_solution)

    results = {}

    for i, city in enumerate(self.cities):
        price_adjustment = optimal_solution[i * 3]
        period_extension = optimal_solution[i * 3 + 1]
        government_investment = optimal_solution[i * 3 + 2]
```

```
base_data = self.base_data[city]

# 计算优化后的指标

new_ticket_price = base_data["基础门票价格(元)"] * (1 + price_adjustment)
price_effect = base_data["价格弹性系数"] * price_adjustment
investment_effect = base_data["游客弹性系数"] * (
    government_investment / base_data["基础设施投入(亿元)"] - 1)

# 应用非线性增长因素

growth_factor = 1 + price_effect + investment_effect
if growth_factor > 1.3:
    growth_factor = 1.3 + 0.5 * (growth_factor - 1.3)

new_visitor_count = base_data["基础游客数量(万人次)"] * growth_factor
ticket_revenue = new_visitor_count * new_ticket_price / 10000
new_period = base_data["当前花期(天)"] + period_extension

# 应用规模效应计算总收入

base_total_revenue = ticket_revenue / base_data["门票收入占比"]
period_effect = 1 + base_data["花期弹性系数"] * period_extension / new_period
scale_factor = min(1.0, 1.2 - 0.1 * np.log(new_visitor_count / 1000))
total_revenue = base_total_revenue * period_effect * scale_factor

# 计算经济效益

economic_benefit = total_revenue - government_investment

results[city] = {
    "花卉类型": self.city_flowers[city],
```

```
"门票价格调整率": f"{price_adjustment:.2%}",
"新门票价格(元)": round(new_ticket_price, 2),
"花期延长(天)": round(period_extension, 1),
"新花期(天)": round(new_period, 1),
"政府投入(亿元)": round(government_investment, 2),
"游客数量(万人次)": round(new_visitor_count, 2),
"门票收入(亿元)": round(ticket_revenue, 2),
"总收入(亿元)": round(total_revenue, 2),
"经济效益(亿元)": round(economic_benefit, 2),
"优化状态": "次优解"
}

print("使用次优解成功!")

return results, optimal_benefit

else:

print("无法获取次优解，返回 None")

return None, None

else:

optimal_solution = result.x
optimal_benefit = -result.fun # 转换回正值

results = {}

for i, city in enumerate(self.cities):
    price_adjustment = optimal_solution[i * 3]
    period_extension = optimal_solution[i * 3 + 1]
    government_investment = optimal_solution[i * 3 + 2]
```

```
base_data = self.base_data[city]

# 计算优化后的指标

new_ticket_price = base_data["基础门票价格(元)"] * (1 + price_adjustment)
price_effect = base_data["价格弹性系数"] * price_adjustment
investment_effect = base_data["游客弹性系数"] * (
    government_investment / base_data["基础设施投入(亿元)"] - 1)

# 应用非线性增长因素

growth_factor = 1 + price_effect + investment_effect
if growth_factor > 1.3:
    growth_factor = 1.3 + 0.5 * (growth_factor - 1.3)

new_visitor_count = base_data["基础游客数量(万人次)"] * growth_factor
ticket_revenue = new_visitor_count * new_ticket_price / 10000
new_period = base_data["当前花期(天)"] + period_extension

# 应用规模效应计算总收入

base_total_revenue = ticket_revenue / base_data["门票收入占比"]
period_effect = 1 + base_data["花期弹性系数"] * period_extension / new_period
scale_factor = min(1.0, 1.2 - 0.1 * np.log(new_visitor_count / 1000))
total_revenue = base_total_revenue * period_effect * scale_factor

# 计算经济效益

economic_benefit = total_revenue - government_investment

results[city] = {
    "花卉类型": self.city_flowers[city],
```

```
"门票价格调整率": f"{price_adjustment:.2%}",
"新门票价格(元)": round(new_ticket_price, 2),
"花期延长(天)": round(period_extension, 1),
"新花期(天)": round(new_period, 1),
"政府投入(亿元)": round(government_investment, 2),
"游客数量(万人次)": round(new_visitor_count, 2),
"门票收入(亿元)": round(ticket_revenue, 2),
"总收入(亿元)": round(total_revenue, 2),
"经济效益(亿元)": round(economic_benefit, 2),
"优化状态": "最优解"
}

return results, optimal_benefit

if __name__ == "__main__":
    # 创建模型实例
    model = FlowerEconomyOptimizationModelV5()

    # 求解优化问题
    print("正在求解优化问题...")
    result = model.optimize()

    # 格式化结果
    print("正在格式化结果...")
    results, total_benefit = model.format_optimization_results(result)

    # 打印结果
    if results:
        print("\n" + "="*50)
        print("花卉经济优化结果汇总")
```

```
print("="*50)

print(f"总经济效益: {total_benefit:.2f} 亿元")

print("="*50)

for city, data in results.items():

    print(f"\n{city} ({data['花卉类型']})")

    print("-"*30)

    print(f"门票价格调整率: {data['门票价格调整率']}")

    print(f"新门票价格: {data['新门票价格(元)']} 元")

    print(f"花期延长: {data['花期延长(天)']} 天")

    print(f"新花期: {data['新花期(天)']} 天")

    print(f"政府投入: {data['政府投入(亿元)']} 亿元")

    print(f"游客数量: {data['游客数量(万人次)']} 万人次")

    print(f"门票收入: {data['门票收入(亿元)']} 亿元")

    print(f"总收入: {data['总收入(亿元)']} 亿元")

    print(f"经济效益: {data['经济效益(亿元)']} 亿元")

    print(f"优化状态: {data['优化状态']}")

else:

    print("未能获得优化结果")
```

表 23 杭州地区天气预测表

时间	降水量（mm）	持续时间（h）	雨纷纷
2025 年 4 月 4 日	0.00	3.08	否
2025 年 4 月 5 日	0.11	0.18	否
2025 年 4 月 6 日	0.32	2.92	否
2026 年 4 月 4 日	0.13	0.5	否
2026 年 4 月 5 日	0.05	1.48	否

2026 年 4 月 6 日	0.00	1.72	否
----------------	------	------	---

表 24 洛阳地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	0.00	0.86	否
2025 年 4 月 5 日	0.00	2.44	否
2025 年 4 月 6 日	0.00	2.58	否
2026 年 4 月 4 日	0.00	2.57	否
2026 年 4 月 5 日	0.00	2.54	否
2026 年 4 月 6 日	0.04	3.22	否

表 25 吐鲁番地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	0.00	0.00	否
2025 年 4 月 5 日	0.00	0.00	否
2025 年 4 月 6 日	0.00	0.00	否
2026 年 4 月 4 日	0.00	0.00	否
2026 年 4 月 5 日	0.00	0.00	否
2026 年 4 月 6 日	0.00	0.00	否

表 26 武汉地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	0.62	1.28	否
2025 年 4 月 5 日	1.36	0.00	否
2025 年 4 月 6 日	0.57	3.31	是
2026 年 4 月 4 日	0.91	2.27	否
2026 年 4 月 5 日	2.09	4.67	是
2026 年 4 月 6 日	0.00	6.14	否

表 27 婺源地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	1.10	7.94	是
2025 年 4 月 5 日	0.56	8.21	是
2025 年 4 月 6 日	0.23	5.89	是
2026 年 4 月 4 日	1.12	12.25	是
2026 年 4 月 5 日	0.12	4.81	是
2026 年 4 月 6 日	1.06	8.97	是

表 28 西安地区天气预测表

时间	降水量 (mm)	持续时间 (h)	雨纷纷
2025 年 4 月 4 日	3.17	2.28	否
2025 年 4 月 5 日	0.00	3.31	否
2025 年 4 月 6 日	0.19	0.04	否
2026 年 4 月 4 日	5.08	9.18	是
2026 年 4 月 5 日	0.91	9.74	是
2026 年 4 月 6 日	0.00	8.91	否